Software Engineering Project Report

HouseMate

A comprehensive client-server application for shared households

---

Developed by:

Mattia Amati, Simone Paudice, Tommaso Nencioni

Teacher supervisor:
Enrico Vicario

Course:
Software Engineering

University of Florence - DINFO
Academic Year 2025/2026
March 10, 2026

Abstract

This report presents the engineering process, architectural decisions and implementation details of **HouseMate**, a client-server application developed to solve the logistical and financial challenges of shared living. Managing shared expenses, assigning chores fairly and coordinating shared shopping lists are common friction points in households. HouseMate addresses these problems by providing a unified platform where members can view real-time data, settle debts easily and transparently assign responsibilities.

The software system comprises a Java Spring Boot backend, a JavaFX-based client and a shared library for intra-module communication.

Contents

List of Figures

1. Introduction

1.1 Overview

Living in a shared apartment or house involves numerous responsibilities that are shared among housemates, including tracking expenses, paying bills, buying groceries, and performing chores. *HouseMate* is a software solution designed to streamline the management of these tasks. The application aims to reduce disputes and organizational overhead by providing a single, reliable point of truth shared between roommates.

1.2 Statement of Problems and Solutions

In many shared living environments, relying on group chats and personal notes to manage household commitments is the standard. While fast and immediate, this decentralized approach inevitably leads to unnecessary stress, wasted time, and avoidable conflicts among roommates.

HouseMate is designed to eliminate this friction. By digitalizing the household management ecosystem, it provides an automated, centralized platform that directly targets and resolves the most common pain points of sharing an apartment:

1. **Financial friction**
   The Problem: Lost receipts, forgotten verbal agreements and the unequal division of bills create frustrating financial inconsistencies and tensions.
   The Solution: **Centralized expense and debt tracking.** HouseMate acts as an impartial financial ledger. Residents insert transactions and the system logs all shared expenses and automatically calculates each roommate's individual debts and monthly contributions, ensuring absolute fairness without awkward conversations.

2. **Division of chores**
   The Problem: The lack of a clear and visible schedule often results in an unequal distribution of household chores, leaving a few members to do most of the work.
   The Solution: **Smart chore management.** This module allows flatmates to seamlessly define and assign specific tasks to each other. It tracks completion statuses in real time and automatically flags overdue chores, keeping the

whole house accountable and the environment clean.

3. **Grocery shopping**
   The Problem: Disorganized communication leads to highly inefficient grocery runs where essential items are either forgotten entirely or accidentally purchased multiple times by multiple people.
   The Solution: **Real-time shopping lists.** HouseMate provides dynamic, shared shopping lists that users can customize at will. Anyone at the store can see exactly what the house needs at all times, without guessing and wasting money.

By directly pairing these everyday hurdles with intuitive and automated solutions, HouseMate transforms a stressful shared apartment into a seamlessly managed home.

1.3 Use of Artificial Intelligence

During the development of *HouseMate*, Artificial Intelligence tools were utilized to assist with repetitive coding tasks and to generate the final styled versions of architectural diagrams based on our detailed structure previously built before and during development. It is important to emphasize that all AI-generated outputs were strictly supervised, reviewed, and refined under careful human control to ensure accuracy and strict alignment with the project requirements.

2. Requirements Analysis

To construct a valuable tool for shared living, a rigorous requirements analysis was conducted. The requirements are broken down into Functional Requirements (actions the system must perform) and Non-Functional Requirements (qualities the system must possess).

2.1 Functional Requirements

The functional requirements are structured around the core modules of the application: user authentication and profile management, household management, financial tracking, chore coordination and shopping list management.

2.1.1 User Authentication and Profile Management

- **Registration & Login (FR-1):** The system shall allow users to register an account by providing a name, surname, email, and password. Upon successful registration, the system must return a valid authentication token, allowing immediate access within the session. Registered users must be able to log in using their email and password.
- **Profile Management (FR-2):** Authenticated users must be able to view and update their profile information, including optional financial details such as IBAN and a payment link, to facilitate real-world debt settlements among housemates.

2.1.2 Household Management

- **Create Household (FR-3):** An authenticated user who does not already belong to a household must be able to create a new one by specifying a unique household name. The creating user is automatically assigned the administrator role for that household, and a unique invitation code is generated upon creation.
- **Join Household (FR-4):** A user who does not belong to any household must be able to join an existing one by providing a valid invitation code. The user is added as a standard (non-admin) member.

- **Members Overview (FR-5):** Users must be able to view the full list of current members belonging to their household, including each member's profile information and administrative role status.
- **Invitation Code Management (FR-6):** Users must be able to retrieve the current invitation code for their household. Household administrators must be able to refresh the invitation code, invalidating the previous one to control access.
- **Remove Member (FR-7):** A household administrator must be able to remove any of its members.
- **Leave Household (FR-8):** A user must be able to voluntarily leave their current household. If the leaving user is the sole administrator and other members remain, the system must automatically transfer the admin role to another member. If the leaving user is the last member, the household is deleted.

2.1.3 Financial Tracking

- **Log Expense (FR-9):** A household member shall be able to register a new expense by specifying a description, a total amount, the list of involved household members and a split type (which can be equal or unequal). The system must compute each user's individual share according to the selected split strategy and persist the expense.
- **Expenses Overview (FR-10):** The system must provide a household-level expense overview reporting the total amount and count of expenses for the current calendar month, also providing support for dynamic expense filtering. Additionally, users must be able to retrieve a personal net cash-flow summary for the current month.
- **Automatic Debt Calculation (FR-11):** Upon logging an expense, the system must automatically compute and update the debt ledger, creating or adjusting debt records between the payer and each participant via a netting algorithm that simplifies complex interdependent debts into streamlined balances.
- **Debts Overview (FR-12):** Users must be able to view their pending debts, filtered by transaction role (creditor, debtor or both) and, optionally, by counterpart. The system must also provide an aggregate overview showing the total amounts owed by and to the user.

- **Settle Debt (FR-13):** A user who has debts with another user must be able to log a settlement payment specifying the settled amount and an optional description. The system must validate that the settlement amount does not exceed the current debt balance and reduce the balance accordingly.
- **Settlements Overview (FR-14):** Users must be able to view the history of settlement transactions within their household, with support for dynamic filtering.

2.1.4 Chore and Task Management

- **Create Chore (FR-15):** A household member must be able to define a new chore by specifying a description and a recurrence frequency in days.
- **Assign Chore (FR-16):** Users must be able to create an assignment for an existing chore, specifying the assigned user and an optional due date.
- **Chores Overview (FR-17):** Users must be able to view all chores defined within their household and retrieve filtered lists of chore assignments. The system must also provide an assignment overview summarizing total, completed, pending, and overdue counts both at the household level and for the individual user.
- **Delete Chore (FR-18):** Users must be able to delete a chore (which cascades the removal of all its assignments) or delete an individual assignment.

2.1.5 Shopping List Management

- **Create Shopping List (FR-19):** A household member must be able to create a shared shopping list containing a set of items.
- **Update Shopping List (FR-20):** Users must be able to mark individual items within a shopping list as bought or not bought, and also modify the items list content itself. The system must automatically update the list's overall status: `NOT_STARTED` if no items are bought, `IN_PROGRESS` if some items are bought, and `COMPLETED` if all items are bought.
- **Shopping Lists Overview (FR-21):** Users must be able to retrieve all shopping lists belonging to their household, including each list's name, items with their bought status, overall list status, and creation date.

- **Delete Shopping List (FR-22):** Users must be able to delete a shopping list from their household.

2.2 Non-Functional Requirements

- **Security (NFR-1):** All API endpoints, except registration and login, must be protected from unauthorized accesses using JSON Web Tokens (JWT). The system must verify the token's validity and extract the user identity on every request via a dedicated authentication filter. User passwords must be hashed using a strong one-way algorithm (BCrypt) before being persisted to the database.
- **Maintainability (NFR-2):** The system must be structured following established object-oriented design principles to allow future extension of capabilities with minimal coupling. The codebase must separate concerns into distinct layers: controllers for REST API HTTP requests handling, backend services for business logic and frontend clients to translate users actions into HTTP requests. While the primary client is a JavaFX desktop application, the API-first design must allow the future development of web or mobile clients without backend modifications.
- **Reliability (NFR-3):** The backend must handle database transaction errors gracefully by employing Spring's `@Transactional` annotation to ensure atomicity (either all needed records are persisted or none are, preventing corrupted household states).
- **Data Validation (NFR-4):** All incoming external data must be validated at the API controller level using Jakarta Bean Validation annotations. This ensures that invalid or malicious input is rejected with informative error responses before reaching the service layer. To mitigate risks from internal service-to-service communication, which may bypass the API boundary, each backend service must independently re-validate its inputs. This two-layer security approach guarantees that every component remains resilient against invalid or malicious data, regardless of whether the request originates from an external client or an internal system.
- **Logging and Observability (NFR-5):** The system must log all significant operations (resource creation, deletion, authentication attempts and errors) at the appropriate severity level, using a structured logging framework, to support debugging and operational monitoring.

3. Use Case Diagrams and Templates

This chapter provides an extensive analysis of the use cases derived from the requirements specified previously, divided in core modules.

3.1 User Authentication and Profile Management

3.1.1 Use Case Diagram

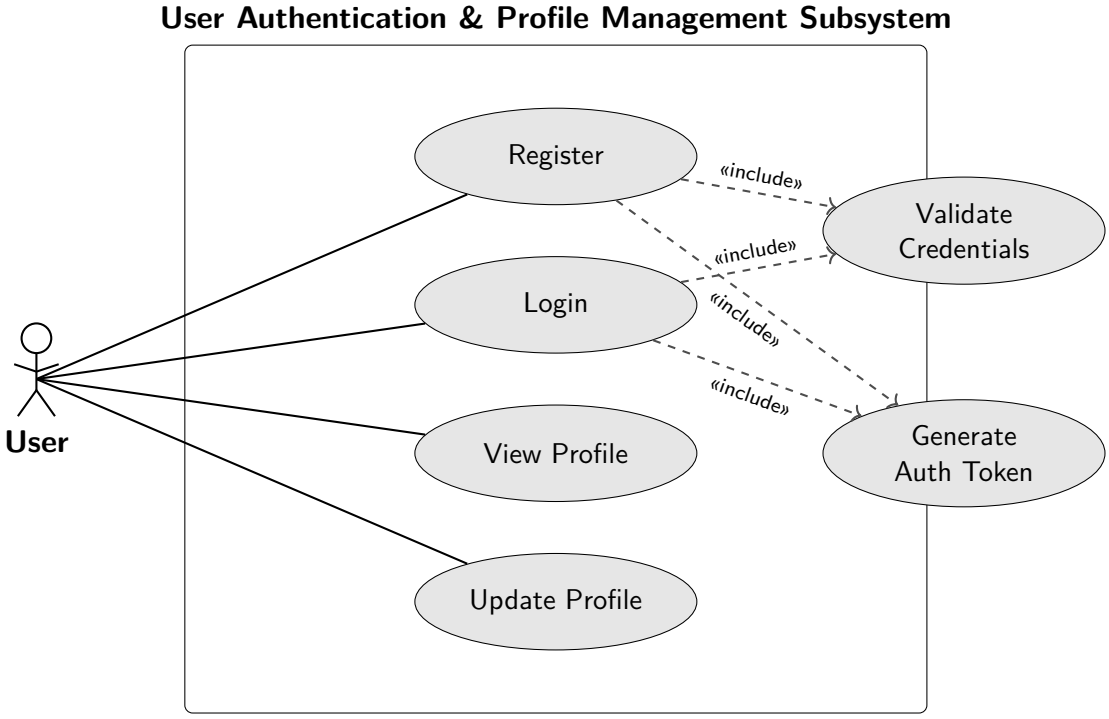

Figure 3.1: Use Case Diagram: User Authentication & Profile Management Subsystem

The diagram above illustrates the use cases composing the user authentication and profile management subsystem. The *User* actor interacts directly with four boundary use cases. The two internal use cases, *Validate Credentials* and *Generate*

Auth Token, are included automatically by the system during registration and login respectively, and are never invoked directly by the user. *View Profile* and *Update Profile* require a valid authentication token.

3.1.2 Use Case Templates

Register UC Template

UC #1: Register

Level	User Goal
Description	Create a new user account and receive an authentication token for immediate access.
Actors	Unregistered User
Pre-conditions	The user does not have an existing account.
Steps	1. The user provides a name, surname, email address, password, and optionally an IBAN. 2. The system validates that all required fields are non-empty and that the email format is valid (include: *Validate Credentials*). 3. The system verifies that the email is not already registered. 4. The system hashes the password and persists the new user record. 5. The system generates an authentication token for the newly created user (include: *Generate Auth Token*). 6. The system returns the user profile and the authentication token.
Post-conditions	A new user account is persisted and a valid authentication token is returned.
Alternative Steps	**A1.** The email is already registered. The system rejects the request. **A2.** Required fields are missing or the email format is invalid. The system rejects the request. **A3.** An IBAN is provided but its format is invalid. The system rejects the request.

Login UC Template

UC #2: Login

Level	User Goal
Description	Authenticate with existing credentials and obtain a new session token.
Actors	Registered User
Pre-conditions	The user has a registered account.
Steps	1. The user provides their email address and password. 2. The system validates that the fields are non-empty and that the email format is valid (include: *Validate Credentials*). 3. The system authenticates the credentials against the stored user record. 4. The system generates a new authentication token (include: *Generate Auth Token*). 5. The system returns the user profile and the authentication token.
Post-conditions	The user receives a valid authentication token for the current session.
Alternative Steps	**A1.** The email does not match any registered account. The system rejects the request. **A2.** The password does not match the stored one. The system rejects the request.

Manage Profile UC Template

UC #3: Manage Profile

Level	User Goal
Description	View and update personal profile information.
Actors	Authenticated User
Pre-conditions	The actor must be authenticated with a valid session token.

Steps	1. The actor requests their current profile information. 2. The system retrieves and returns the actor's profile data including name, surname, email, IBAN, and payment link. 3. The actor provides updated profile fields (name, surname, email, IBAN, or payment link). 4. The system validates the updated fields and persists the changes. 5. The system returns the updated profile.
Post-conditions	The actor's profile is updated with the new values.
Alternative Steps	**A1.** The actor only views their profile without making changes. The flow ends at step 2. **A2.** Any of the provided data for profile update has an invalid format. The system rejects the update request.

3.2 Household Management

3.2.1 Use Case Diagram

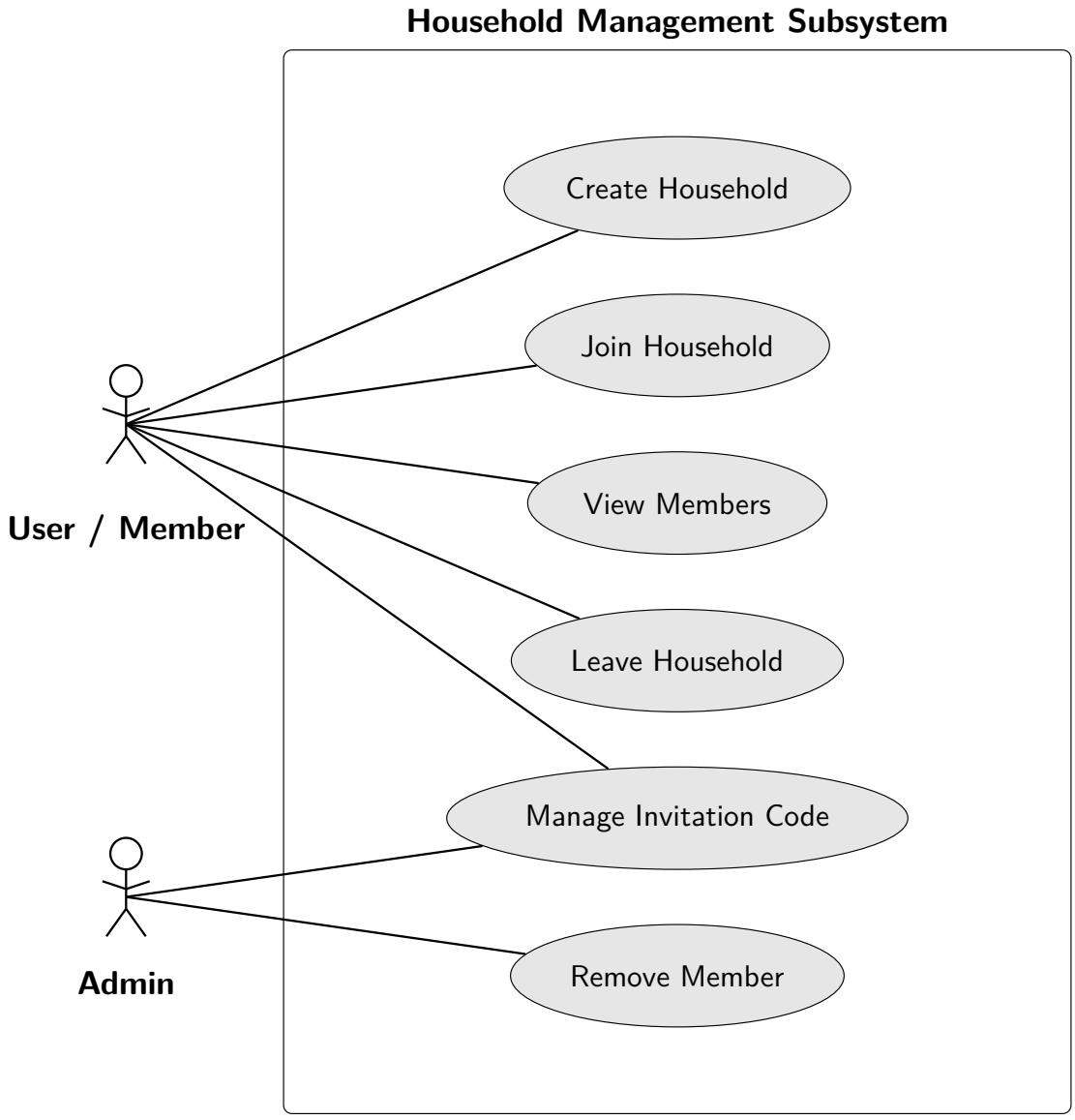

Figure 3.2: Use Case Diagram: Household Management Subsystem

The diagram above illustrates the use cases composing the household management subsystem. The *User / Member* actor represents both an authenticated user without a household (interacting with creation and join use cases) and a standard household member (viewing members, leaving, and retrieving the invitation code of their household). The *Admin* actor represents a household administrator with elevated privileges, capable of refreshing the invitation code and removing other members.

3.2.2 Use Case Templates

Create Household UC Template

UC #1: Create Household

Level	User Goal
Description	Create a new household and automatically become its administrator.
Actors	Authenticated User
Pre-conditions	The user is authenticated and does not already belong to a household.
Steps	1. The user specifies a unique name for the new household. 2. The system validates that the name is non-empty and not already in use. 3. The system generates a unique invitation code for the household. 4. The system persists the household and adds the user as its only member and administrator. 5. The system returns the household details.
Post-conditions	A new household is persisted, and the user is assigned the administrator role.
Alternative Steps	**A1.** The user already belongs to a household. The system rejects the request. **A2.** The chosen household name already exists. The system rejects the request.

Join Household UC Template

UC #2: Join Household

Level	User Goal
Description	Join an existing household using a valid invitation code.
Actors	Authenticated User
Pre-conditions	The user is authenticated and does not already belong to a household.
Steps	1. The user provides a household invitation code. 2. The system validates the code against existing households. 3. The system adds the user to the household as a standard member. 4. The system returns the household details.
Post-conditions	The user is added to the household as a standard member.
Alternative Steps	**A1.** The user already belongs to a household. The system rejects the request. **A2.** The invitation code is invalid or expired. The system rejects the request.

View Members UC Template

UC #3: View Members

Level	User Goal
Description	Retrieve the list of all members in the current household.
Actors	Household Member
Pre-conditions	The user must be authenticated and belong to an active household.
Steps	1. The user requests the list of household members. 2. The system retrieves all members of the household along with their roles (admin or standard) and profile information. 3. The system returns the list of members.
Post-conditions	The list of household members is returned to the user.
Alternative Steps	**A1.** The user does not belong to a household. The system rejects the request.

Manage Invitation Code UC Template

UC #4: Manage Invitation Code

Level	User Goal
Description	Retrieve the current invitation code or generate a new one to control household access.
Actors	Household Member, Household Admin
Pre-conditions	The user must be authenticated and belong to an active household.
Steps	1. The user requests to view the household's current invitation code. 2. The system retrieves and returns the current valid invitation code. 3. (Admin only) The administrator requests to refresh the invitation code. 4. (Admin only) The system generates a new unique invitation code, invalidating the previous one. 5. (Admin only) The system returns the newly generated invitation code.
Post-conditions	The current or newly generated invitation code is returned to the user.
Alternative Steps	**A1.** A non-admin member attempts to refresh the code. The system rejects the request.

Remove Member UC Template

UC #5: Remove Member

Level	User Goal
Description	Remove a member from the household.
Actors	Household Admin
Pre-conditions	The user must be an administrator of the household.

Steps	1. The administrator selects a member to remove from the household. 2. The system validates that the requester is an admin and the target user is a member of the same household. 3. The system verifies that the administrator is not attempting to remove themselves. 4. The system removes the target user from the household. 5. The system returns the updated household details.
Post-conditions	The target user is removed from the household.
Alternative Steps	**A1.** The requester is not an admin. The system rejects the request. **A2.** The administrator attempts to remove themselves. The system instructs them to use the "Leave Household" flow instead.

Leave Household UC Template

UC #6: Leave Household

Level	User Goal
Description	Voluntarily leave the current household.
Actors	Household Member
Pre-conditions	The user must be authenticated and belong to an active household.
Steps	1. The user requests to leave their current household. 2. The system removes the user from the household members list. 3. If the user was the sole administrator and other members remain, the system automatically transfers the admin role to another member. 4. If the leaving user was the last member, the system deletes the household. 5. The system confirms the successful departure.
Post-conditions	The user is removed from the household. The household may be deleted or have its admin reassigned if necessary.

Alternative Steps	**A1.** The user does not belong to a household. The system rejects the request.

3.3 Financial Tracking

3.3.1 Use Case Diagram

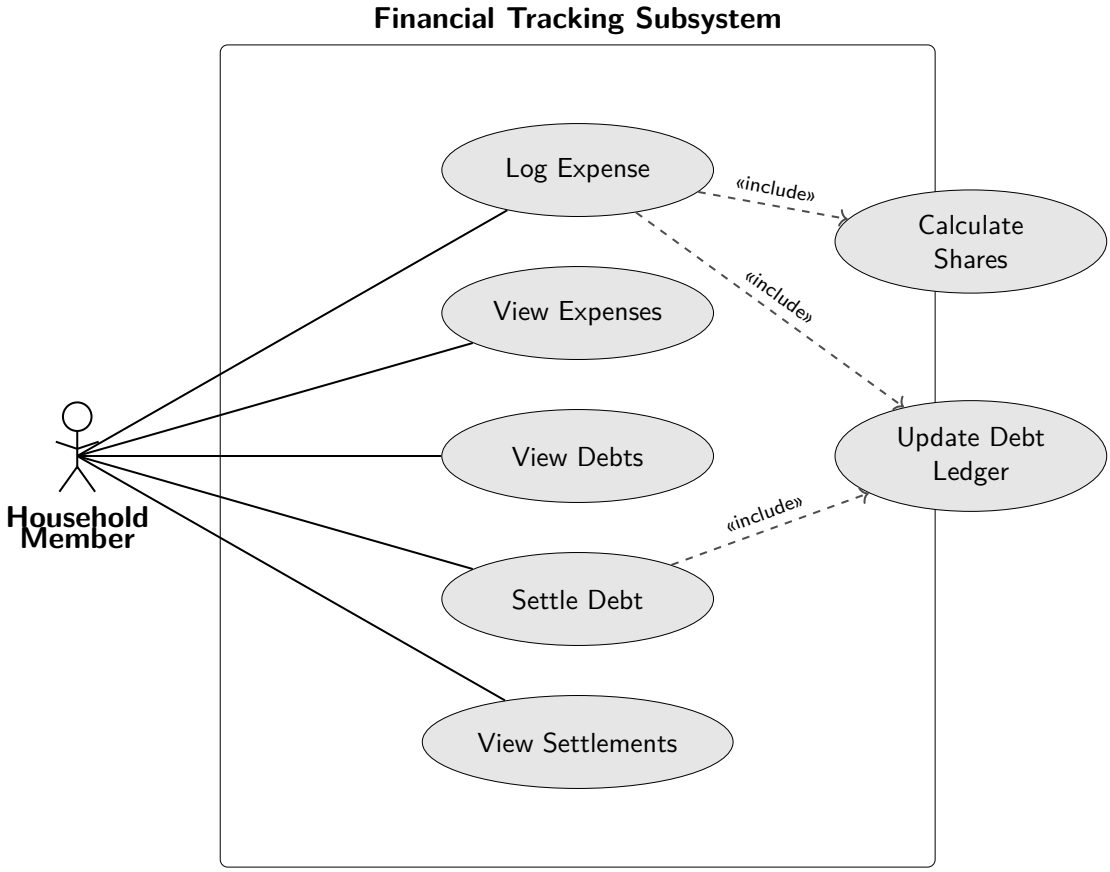

Figure 3.3: Use Case Diagram: Financial Tracking Subsystem

The diagram above illustrates the use cases composing the expense management subsystem. The *Household Member* actor interacts directly with five boundary use cases. The two internal use cases, *Calculate Shares* and *Update Debt Ledger*, are

included automatically by the system during expense creation and debt settlement respectively, and are never invoked directly by the user.

3.3.2 Use Case Templates

Log Expense UC Template

UC #1: Log Expense

Level	User Goal
Description	Record a shared expense and automatically update all affected debts.
Actors	Household Member
Pre-conditions	The actor must be authenticated and belong to an active household.
Steps	1. The actor provides a description, total amount, a split type (equal, proportional, exact amounts, or adjustment), and the list of involved members. 2. The system validates that the amount is positive and the participant list is non-empty. 3. The system selects the appropriate splitting strategy and computes each member's share (include: *Calculate Shares*). 4. The expense and all individual shares are persisted atomically. 5. For each participant, the system updates the debt ledger (include: *Update Debt Ledger*). 6. The system returns the created expense with all computed shares.
Post-conditions	The expense record and its computed shares are persisted. All affected debt balances are updated via the netting algorithm.
Alternative Steps	**A1.** If the split configuration is invalid (e.g., exact amounts do not sum to the total), the system rejects the request and no data is persisted. **A2.** If a participant does not exist in the system, the entire operation is rolled back.

View Expenses UC Template

UC #2: View Expenses

Level	User Goal
Description	Retrieve a filtered list or an aggregated overview of household expenses.
Actors	Household Member
Pre-conditions	The actor must be authenticated and belong to an active household.
Steps	1. The actor requests the expense list, optionally providing filters: transaction role, date range, and description keyword. 2. The system constructs a dynamic query from the provided filters, scoped to the actor's household. 3. The system returns the matching expenses, each including the payer's identity and the individual shares breakdown.
Post-conditions	The requested expense data is returned to the actor.
Alternative Steps	**A1.** The actor requests the household expense summary instead. The system returns the total amount and count of expenses for the current month. **A2.** The actor requests their personal financial summary instead. The system computes the net cash-flow for the current month and returns the aggregate value.

View Debts UC Template

UC #3: View Debts

Level	User Goal
Description	Retrieve a filtered list or an aggregated overview of the actor's debts.
Actors	Household Member
Pre-conditions	The actor must be authenticated and belong to an active household.

Steps	1. The actor requests their debts, specifying a role filter (creditor, debtor, or both) and an optional counterpart filter. 2. The system queries the debt ledger for the actor's household, applying the requested filters. 3. The system returns each matching debt with the counterpart's identity, the actor's role, and the outstanding amount.
Post-conditions	The requested debt data is returned to the actor.
Alternative Steps	**A1.** The actor requests the debt summary instead. The system computes and returns the aggregate totals of amounts owed by and to the actor.

Settle Debt UC Template

UC #4: Settle Debt

Level	User Goal
Description	Record a payment that reduces or eliminates an outstanding debt.
Actors	Household Member
Pre-conditions	The actor must be authenticated. An outstanding debt must exist where the actor is the debtor.
Steps	1. The actor selects an outstanding debt and provides a settlement amount and an optional description. 2. The system validates that the actor is the debtor, that the creditor matches, and that the amount does not exceed the current balance. 3. The system creates a settlement record and reduces the debt balance accordingly (include: *Update Debt Ledger*). 4. If the debt is fully covered, the balance is set to zero rather than deleting the record. 5. The system returns the created settlement record.

Post-conditions	A settlement record is persisted and the debt balance is reduced. If fully settled, the balance is set to zero.
Alternative Steps	**A1.** The actor is not the debtor on the selected debt. The system rejects the request. **A2.** The specified creditor does not match the debt record. The system rejects the request. **A3.** The settlement amount exceeds the current balance. The system rejects the request to prevent negative balances.

View Settlements UC Template

UC #5: View Settlements

Level	User Goal
Description	Retrieve a filtered list of settlement transactions within the household.
Actors	Household Member
Pre-conditions	The actor must be authenticated and belong to an active household.
Steps	1. The actor requests the settlement history, optionally providing filters: transaction role, date range, and description keyword. 2. The system constructs a dynamic query from the provided filters, scoped to the actor's household. 3. The system returns the matching settlements, each including the counterpart's identity, the settled amount, timestamp, and description.
Post-conditions	The requested settlement data is returned to the actor.
Alternative Steps	**A1.** No settlements match the provided filters. The system returns an empty list.

3.4 Shopping List Management

3.4.1 Use Case Diagram

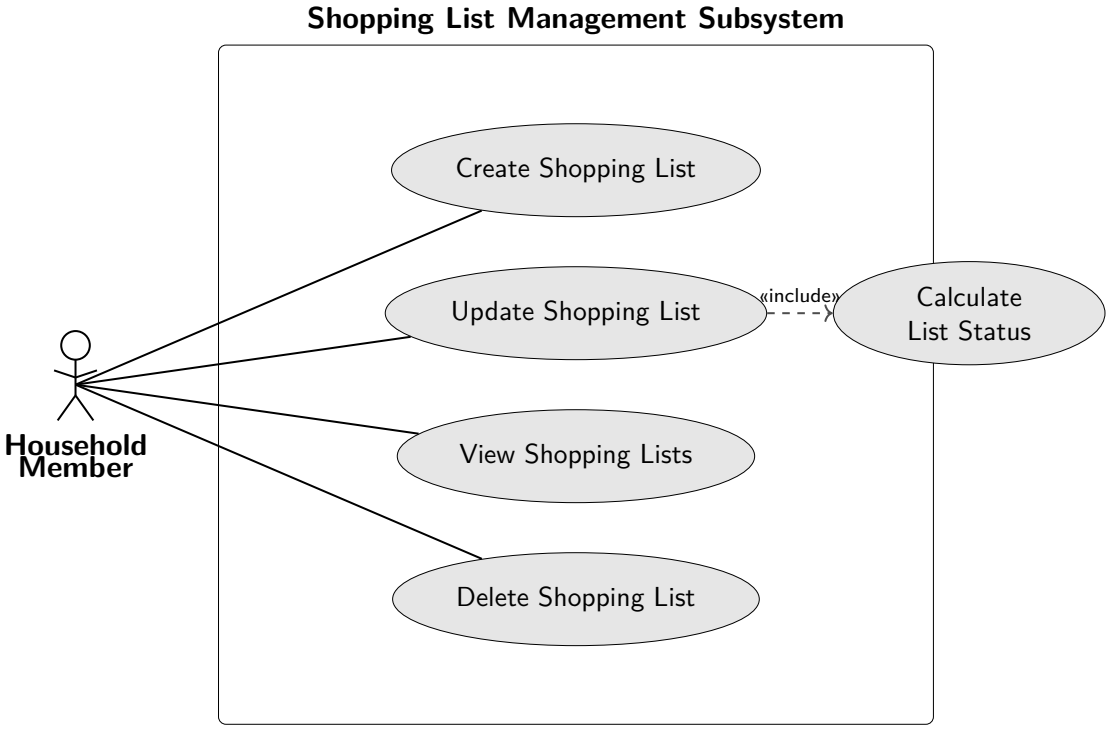

Figure 3.4: Use Case Diagram: Shopping List Management Subsystem

The diagram above illustrates the use cases composing the shopping list management subsystem. The *Household Member* actor interacts directly with four boundary use cases for full CRUD capabilities on shared shopping lists. The internal use case, *Calculate List Status*, is included automatically by the system during list updates to determine if the list is completely bought, partially bought, or not started.

3.4.2 Use Case Templates

Create Shopping List UC Template

UC #1: Create Shopping List

Level	User Goal
Description	Create a new shared shopping list for the household.
Actors	Household Member
Pre-conditions	The user must be authenticated and belong to an active household.
Steps	1. The user provides a name and a set of items to buy. 2. The system creates a new shopping list associated with the user's household, initialized with a `NOT_STARTED` status. 3. The system persists the shopping list and returns its details.
Post-conditions	A new shopping list is persisted and available to the household.

Update Shopping List UC Template

UC #2: Update Shopping List

Level	User Goal
Description	Mark items as bought or modify the contents of an existing shopping list.
Actors	Household Member
Pre-conditions	The user must be authenticated, belong to an active household, and the target shopping list must exist.
Steps	1. The user selects a shopping list and provides an updated array of item statuses (bought or not bought). 2. The system validates that the user belongs to the household owning the list. 3. The system applies the bought statuses to the list items. 4. The system calculates the overall list status: `NOT_STARTED` if none are bought, `COMPLETED` if all are bought, `IN_PROGRESS` otherwise (include: *Calculate List Status*). 5. The system persists the changes and returns the updated shopping list.
Post-conditions	The shopping list items and overall status are updated.

Alternative Steps	**A1.** The user does not belong to the household that owns the list. The system rejects the request.

View Shopping Lists UC Template

UC #3: View Shopping Lists

Level	User Goal
Description	Retrieve the overview of all shopping lists within the household.
Actors	Household Member
Pre-conditions	The user must be authenticated and belong to an active household.
Steps	1. The user requests the overview of their household's shopping lists. 2. The system retrieves all shopping lists associated with the household, including their items and current overall statuses. 3. The system returns the lists to the user.
Post-conditions	The user receives the shopping lists data.

Delete Shopping List UC Template

UC #4: Delete Shopping List

Level	User Goal
Description	Permanently delete a shopping list from the household.
Actors	Household Member
Pre-conditions	The user must be authenticated, belong to an active household, and the target shopping list must exist.
Steps	1. The user selects a shopping list for deletion. 2. The system validates that the user belongs to the household owning the list. 3. The system removes the shopping list from the database. 4. The system confirms the successful deletion.

Post-conditions	The shopping list is deleted from the system.
Alternative Steps	**A1.** The user does not belong to the household that owns the list. The system rejects the request.

4. System Architecture

4.1 High-Level Architecture

The system is designed around a decoupled Client-Server architecture, logically partitioned into three Maven modules:

- **Backend:** Contains the server logic, built upon the Spring Boot framework.
- **Client:** Contains the consumer JavaFX desktop application.
- **Shared:** Contains standard enums, utilities and Data Transfer Objects (DTOs) common to both the server and the client. In addition to avoiding code repetition, this module guarantees both communication sides serialize and deserialize payloads using the exact same class definitions, ensuring strict network type-safety.

Such a structure ensures the backend remains entirely agnostic to the presentation layer, relying strictly on standard data-exchange formats (JSON) over the HTTP protocol.

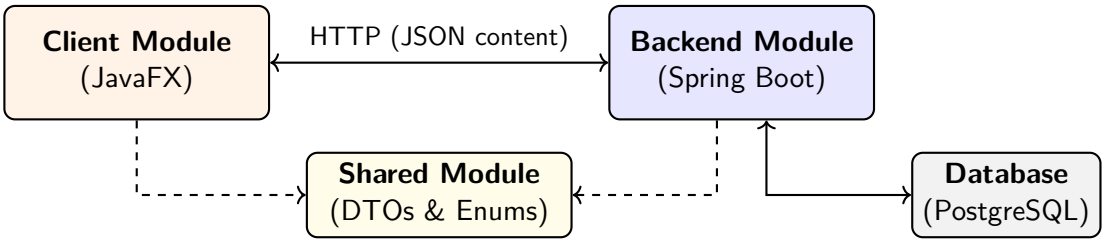

Figure 4.1: High-Level Modular Architecture

4.2 Technological Stack

The application leverages **Java 17+** and relies on the following core technologies to fulfill its functional and non-functional requirements:

- **Server:** *Spring Boot* provides the foundational inversion of control and auto-configuration. Routing is handled by *Spring Web MVC*, while *Spring Security* enforces JWT-based authentication. The data layer utilizes *Spring Data JPA* and *Hibernate* for Object-Relational Mapping (ORM) against a relational database (*PostgreSQL* / *H2* (for testing)).

- **Client:** *JavaFX* serves as the framework for the Graphical User Interface.
- **Tooling:** *Apache Maven* coordinates multi-module dependency management and the build lifecycle. *JUnit 5* and *Mockito* facilitate unit testing and test-driven development methodologies.

4.3 Backend Architecture

The server implements a strict 3-tier logical layered architecture, which separates concerns and isolates business logic from the network protocols.

1. **Controller Layer:** Handles incoming HTTP connections, maps REST endpoints, and deserializes JSON requests. It delegates instructions to the underlying services, then serializes the results into a JSON response that gets sent back to the requesting client.
2. **Service Layer:** This tier encapsulates the core business rules (e.g. debt reduction algorithms). It performs strict validation and defines transactional boundaries to guarantee data atomicity and avoid corruption.
3. **Data Access Layer (Repository):** Contains Spring Data JPA interfaces that abstract raw SQL, handling complex queries that map database tables directly to standard Java Objects.

4.4 Client Architecture

The client module implements an API-first architectural design. It mirrors the backend services via dedicated client-side classes injected with a centralized HTTP client wrapper. These classes build requests, include the session JWT in the `'Authorization: Bearer'` header and deserialize the JSON server responses. By wrapping the REST communication in these service classes, the JavaFX UI controllers remain completely unaware of the underlying network protocols, adhering strictly to the Single Responsibility Principle.

4.5 Detailed Architecture Diagram

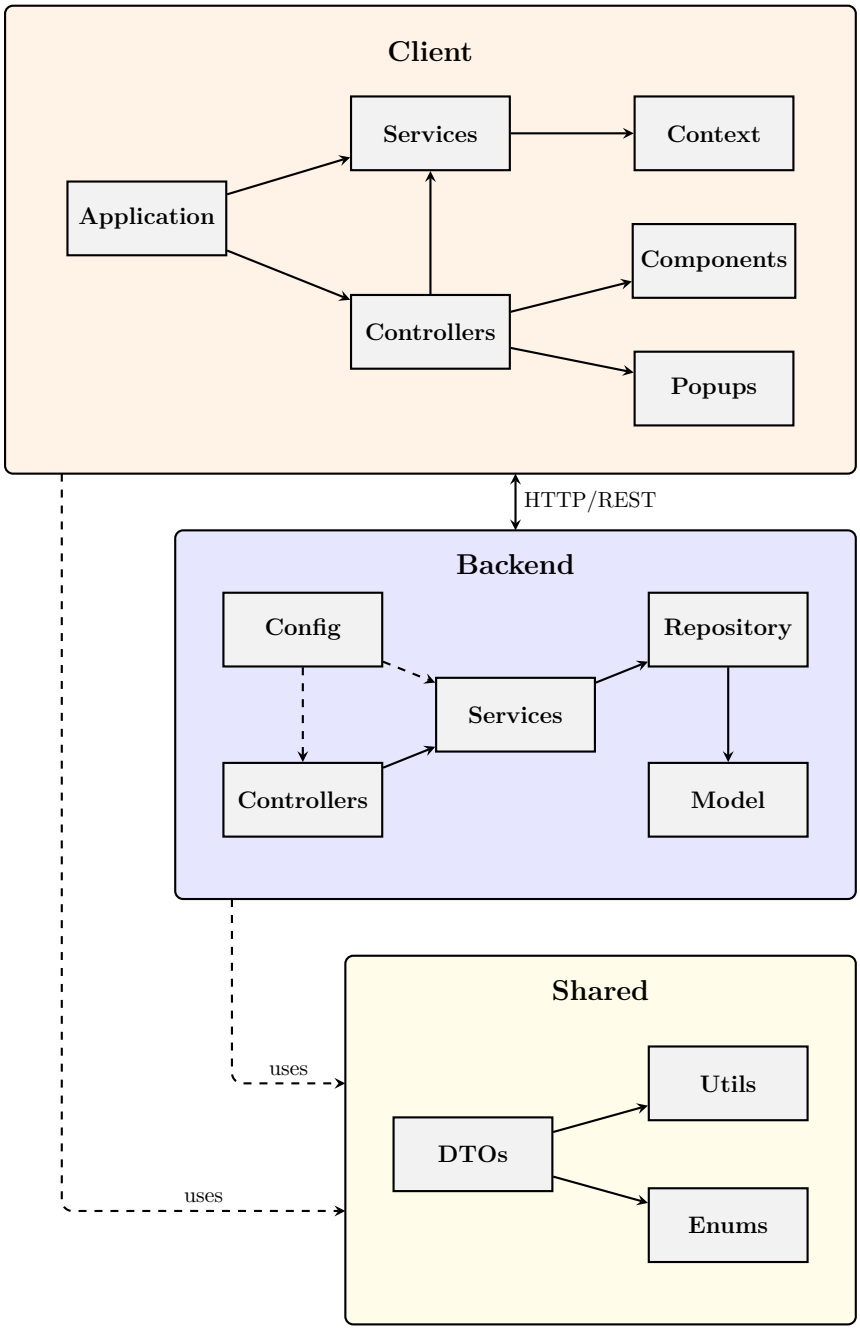

Figure 4.2: Detailed architecture diagram

4.6 Example: User Request Flow

The following diagram illustrates the lifecycle of a user request, tracing its path through the architecture, its interaction with the database and the return of the response back to the user.

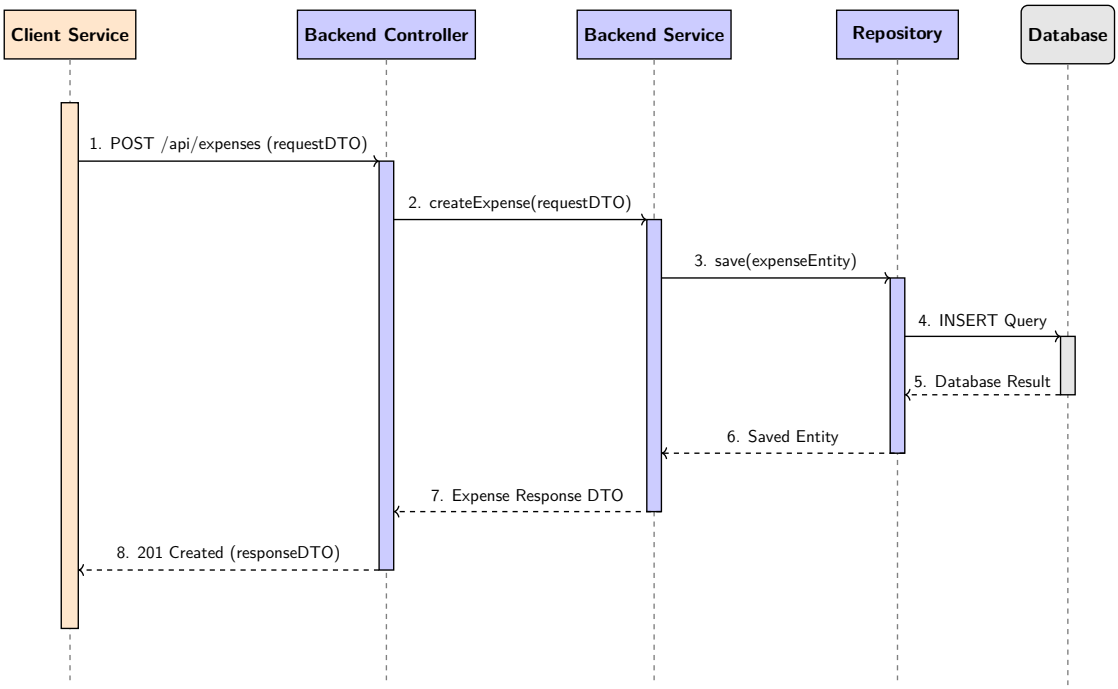

Figure 4.3: User request flow: Adding an Expense

5. API Documentation

5.1 RESTful APIs

The server interacts with the client purely through HTTP protocols. The API design adheres strictly to standard RESTful architectures mapping HTTP verbs directly to CRUD operations on underlying resources. Every protected endpoint receives information about the logged-in client through the `@AuthenticationPrincipal` construct, securely extracting the user identity from the JWT token provided in the HTTP headers.

5.2 Detailed Endpoints Description

5.2.1 Authentication

`POST` `/api/auth/register`
Access: Public
Body: `{ "name": string, "surname": string, "email": string, "password": string, "iban": string? }`
Response: `{ "user": { "id": UUID, "name": string, "surname": string, "email": string, "iban": string?, "paymentLink": string? }, "token": string }`
Description: Accepts user registration details, creates a new account, and returns the user profile along with a JWT session token.

`POST` `/api/auth/login`
Access: Public
Body: `{ "email": string, "password": string }`
Response: `{ "user": { "id": UUID, "name": string, "surname": string, "email": string, "iban": string?, "paymentLink": string? }, "token": string }`
Description: Authenticates a user based on their credentials and returns their profile with a new JWT session token.

5.2.2 Users

`GET` `/api/users/me`
Access: Only logged in users
Response: `{ "id": UUID, "name": string, "surname": string, "email": string, "iban": string?, "paymentLink": string? }`
Description: Retrieves the profile information of the currently authenticated user.

`PATCH` `/api/users/me`
Access: Only logged in users
Body: `{ "name": string?, "surname": string?, email: string?, "iban": string?, "paymentLink": string? }`
Response: `{ "id": UUID, "name": string, "surname": string, "email": string, "iban": string?, "paymentLink": string? }`
Description: Updates specific fields of the current user's profile and returns it with updated data.

5.2.3 Households

`POST` `/api/households`
Access: Only logged in users
Body: `{ "name": string }`
Response: `{ "id": UUID, "name": string, "creationDate": string, "memberships": [ { "user": { "id": UUID, "name": string, "surname": string, "email": string, "iban": string?, "paymentLink": string? }, "membership": { "isAdmin": boolean, "date": string } } ] }`
Description: Instantiates a new household and automatically assigns the creator as the household only member and administrator.

`GET` `/api/households/me`
Access: Only logged in users
Response: `{ "id": UUID, "name": string, "creationDate": string, "memberships": [ { "user": { "id": UUID, "name": string, "surname": string, "email": string, "iban": string?, "paymentLink": string? }, "membership": { "isAdmin": boolean, "date": string } } ] }`
Description: Retrieves details about the user's current household.

`GET` `/api/households/members`
Access: Only logged in users
Response: `[ { "user": { "id": UUID, "name": string, "surname": string, "email": string, "iban": string?, "paymentLink": string? }, "membership": { "isAdmin": boolean, "date": string } } ]`
Description: Retrieves the list of the members of current user's household.

`POST` `/api/households/members`
Access: Only logged in users
Body: `{ "invitationCode": string }`
Response: `{ "id": UUID, "name": string, "creationDate": string, "memberships" [ { "user": { "id": UUID, "name": string, "surname": string, "email": string, "iban": string?, "paymentLink": string? }, "membership": { "isAdmin": boolean, "date": string } } ] }`
Description: Joins an existing household using a valid invitation code.

`DELETE` `/api/households/members/{memberId}`
Access: Only logged in users
Path Params: `memberId` (UUID)
Response: `{ "id": UUID, "name": string, "creationDate": string, "memberships" [ { "user": { "id": UUID, "name": string, "surname": string, "email": string, "iban": string?, "paymentLink": string? }, "membership": { "isAdmin": boolean, "date": string } } ] }`
Description: Removes a specific member from the household. The requester must be an administrator of the household.

`DELETE` `/api/households/me`
Access: Only logged in users
Response: `204 No Content`
Description: Makes the current user leave their household.

`GET` `/api/households/invitation-code`
Access: Only logged in users
Response: `{ "invitationCode": string, "refreshedAt": string }`
Description: Retrieves the current active invitation code for the household.

`POST` `/api/households/invitation-code/refresh`

Access: Only logged in users
Response: `{ "invitationCode": string, "refreshedAt": string }`
Description: Generates a new unique invitation code, invalidating the previous one. The requester must be an administrator of the household.

5.2.4 Expenses

`POST` `/api/expenses`
Access: Only logged in users
Body: `{ "description": string, "amount": decimal, "splitType": string, "shares": [ { "userId": UUID, "share": decimal } ] }`
Response: `{ "id": UUID, "description": string, "date": string, "amount": decimal, "payerId": UUID, "payerFullName": string, "splitType": string, "householdId": UUID, "shares": [ { "id": UUID, "userId": UUID, "userFullName": string, "amount": decimal } ] }`
Description: Registers a new expense, automatically computing each participant's share based on the selected split type, and generates the resulting debt edges.

`GET` `/api/expenses`
Access: Only logged in users
Query Params: `householdId, userTransactionRole, dateRange, description`
Response: `[ { "id": UUID, "description": string, "date": string, "amount": decimal, "payerId": UUID, "payerFullName": string, "splitType": string, "householdId": UUID, "shares": [ { "id": UUID, "userId": UUID, "userFullName": string, "amount": decimal } ] } ]`
Description: Retrieves a filtered list of expenses associated with the requesting user's household.

`GET` `/api/expenses/overview`
Access: Only logged in users
Response: `{ "totalAmount": decimal, "expenseCount": integer }`
Description: Returns aggregated expense statistics (total amount and count) for the current calendar month within the household.

`GET` `/api/expenses/me`
Access: Only logged in users
Response: `{ "actualCashFlowAmount": decimal }`

Description: Returns the net cash flow amount for the requesting user over the current month.

5.2.5 Debts

`GET /api/debts`
Access: Only logged in users
Query Params: `userTransactionRole, involvedId`
Response: `[ { "id": UUID, "debtorId": UUID, "debtorFullName": string, "creditorId": UUID, "creditorFullName": string, "amount": decimal } ]`
Description: Retrieves a filtered list of the user's debt records, either as a creditor or debtor.

`GET /api/debts/me`
Access: Only logged in users
Response: `{ "totalOwedByMe": decimal, "totalOwedToMe": decimal }`
Description: Summarizes the aggregate debt position of the user, showing total amounts owed by and to the user.

`DELETE /api/debts/{debtId}`
Access: Only logged in users
Path Params: `debtId` (UUID)
Response: `204 No Content`
Description: Manually deletes the specified debt record.

5.2.6 Settlements

`POST /api/settlements/{debtId}`
Access: Only logged in users
Path Params: `debtId` (UUID)
Body: `{ "amount": decimal, "description": string? }`
Response: `{ "id": UUID, "debtorId": UUID, "debtorFullName": string, "creditorId": UUID, "creditorFullName": string, "amount": decimal, "description": string, "date": string }`
Description: Creates a settlement transaction for the specified debt, reducing its

outstanding balance accordingly.

`GET /api/settlements`
Access: Only logged in users
Query Params: `householdId, userTransactionRole, dateRange, description`
Response: `[ { "id": UUID, "debtorId": UUID, "debtorFullName": string, "creditorId": UUID, "creditorFullName": string, "amount": decimal, "description": string, "date": string } ]`
Description: Retrieves a filtered list of settlement transactions.

5.2.7 Chores

`POST /api/chores`
Access: Only logged in users
Body: `{ "description": string, "frequencyDays": integer, "householdId": UUID }`
Response: `{ "id": UUID, "description": string, "frequencyDays": integer }`
Description: Defines a new household chore with its description and recurrence parameters.

`POST /api/chores/assignments`
Access: Only logged in users
Body: `{ "choreId": UUID, "assigneeId": UUID, "dueDate": string }`
Response: `{ "assignmentId": UUID, "choreId": UUID, "choreDescription": string, "assignedUser": { "id": UUID, "name": string, "surname": string, "email": string, "iban": string?, "paymentLink": string? }, "dueDate": string, "status": string }`
Description: Creates a specific assignment for an existing chore to a selected user.

`GET /api/chores`
Access: Only logged in users
Response: `[ { "id": UUID, "description": string, "frequencyDays": integer } ]`
Description: Retrieves all base chores defined within the current household.

`GET` `/api/chores/assignments`
Access: Only logged in users
Query Params: `{ "assigneeId": UUID?, "status": string?, "startDate": string?, "endDate": string? }`
Response: `[ { "assignmentId": UUID, "choreId": UUID, "choreDescription": string, "assignedUser": { "id": UUID, "name": string, "surname": string, "email": string, "iban": string?, "paymentLink": string? }, "dueDate": string, "status": string } ]`
Description: Retrieves filtered chore assignments based on status, dates, and assignee.

`GET` `/api/chores/assignments/me`
Access: Only logged in users
Response: `{ "pendingAssignments": integer, "overdueAssignments": integer }`
Description: Returns total pending and overdue assignments counts for the requesting user.

`GET` `/api/chores/assignments/overview`
Access: Only logged in users
Response: `{ "pendingAssignments": integer, "overdueAssignments": integer }`
Description: Returns total pending and overdue assignments counts for the entire household.

`PATCH` `/api/chores/assignments/{assignmentId}/status`
Access: Only logged in users
Path Params: `assignmentId` (UUID)
Body: `{ "status": string }`
Response: `{ "assignmentId": UUID, "choreId": UUID, "choreDescription": string, "assignedUser": { "id": UUID, "name": string, "surname": string, "email": string, "iban": string?, "paymentLink": string? }, "dueDate": string, "status": string }`
Description: Updates the completion status of a specific chore assignment.

`DELETE` `/api/chores/{choreId}`
Access: Only logged in users (Admin)

Path Params: `choreId` (UUID)
Response: `204 No Content`
Description: Deletes a chore and cascades removal to all its associated assignments.

`DELETE` `/api/chores/assignments/{assignmentId}`
Access: Only logged in users (Admin)
Path Params: `assignmentId` (UUID)
Response: `204 No Content`
Description: Deletes a specific chore assignment.

5.2.8 Shopping Lists

`POST` `/api/shopping-lists`
Access: Only logged in users
Body: `{ "name": string, "items": [ { "name": string, "isBought": boolean } ], "householdId": UUID, "creationDate": string }`
Response: `{ "id": UUID, "name": string, "items": [ { "itemName": string, "isBought": boolean } ], "status": string, "householdId": UUID, "creationDate": string }`
Description: Creates a new shared shopping list populated with requested items.

`GET` `/api/shopping-lists`
Access: Only logged in users
Response: `[ { "id": UUID, "name": string, "items": [ { "itemName": string, "isBought": boolean } ], "status": string, "householdId": UUID, "creationDate": string } ]`
Description: Retrieves all shopping lists belonging to the current household.

`PATCH` `/api/shopping-lists/{listId}`
Access: Only logged in users
Path Params: `listId` (UUID)
Body: `{ "boughtItems": [ boolean ] }`
Response: `{ "id": UUID, "name": string, "items": [ { "itemName": string, "isBought": boolean } ], "status": string, "householdId": UUID, "creationDate": string }`
Description: Updates the bought status of items inside the specified shopping list.

`DELETE` `/api/shopping-lists/{listId}`
Access: Only logged in users
Path Params: `listId` (UUID)
Response: `204 No Content`
Description: Permanently removes the specified shopping list from the household.

6. Backend Design and Implementation

6.1 Backend UML Class Diagrams

The backend module section of the report begins with a series of focused UML class diagrams, each highlighting a specific domain area of the implementation to provide both a high-level overview and detailed domain-specific views.

6.1.1 Diagrams Construction

All the UML diagrams that will follow have been created, as mentioned in 1.3, following this pipeline:

- Before and during development, sketches of the structure of the project from a general overview to its detailed components have been created
- Using these sketches as the only references, generative AI has been used to write PlantText code representing the styled diagram.This process followed the same strict-supervised methodology already presented, in terms of AI usage.

6.1.2 High-Level Architecture Overview

This diagram presents a simplified view of the system, showing the main entity classes along with their main attributes and their domain-level relationships. Collapsed sub-domains (Expense Domain, Chore Domain, Shopping List Domain) and the Security Domain are expanded in dedicated diagrams in the following subsections.

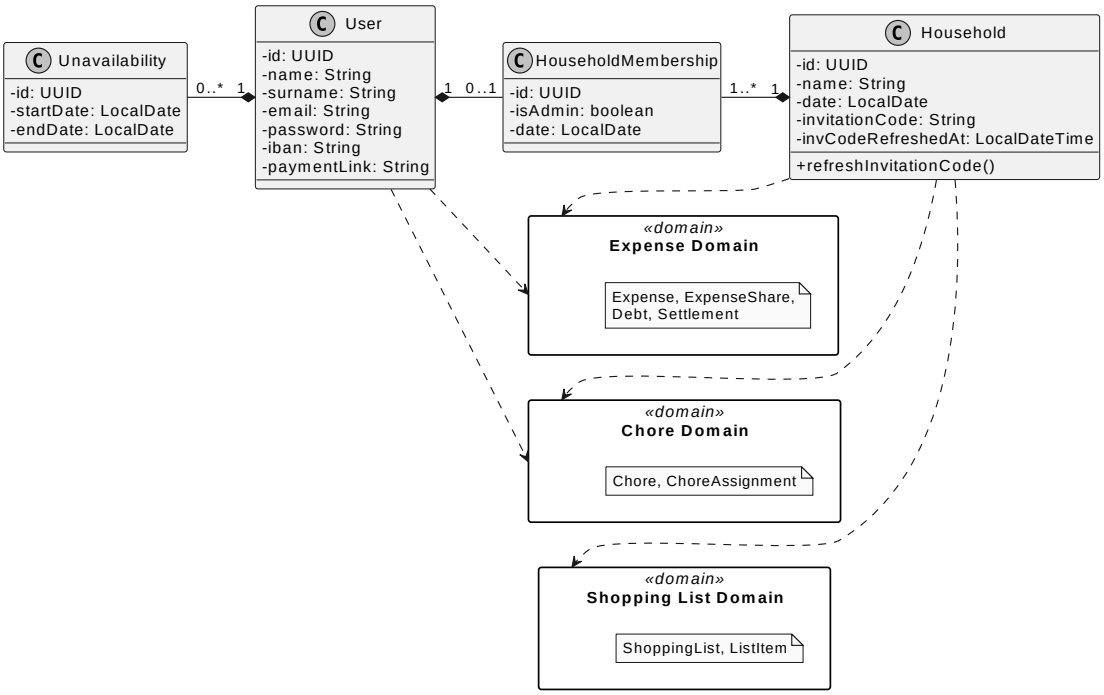

Figure 6.1: High-Level Backend Architecture Overview

6.1.3 Expense Domain Class Diagram

This diagram expands the Expense Domain, showing all classes involved in the financial subsystem: expense recording, share calculation, debt tracking, settlement processing, and the Strategy pattern used for expense splitting, together with their relationships to the core `User` and `Household` entities.

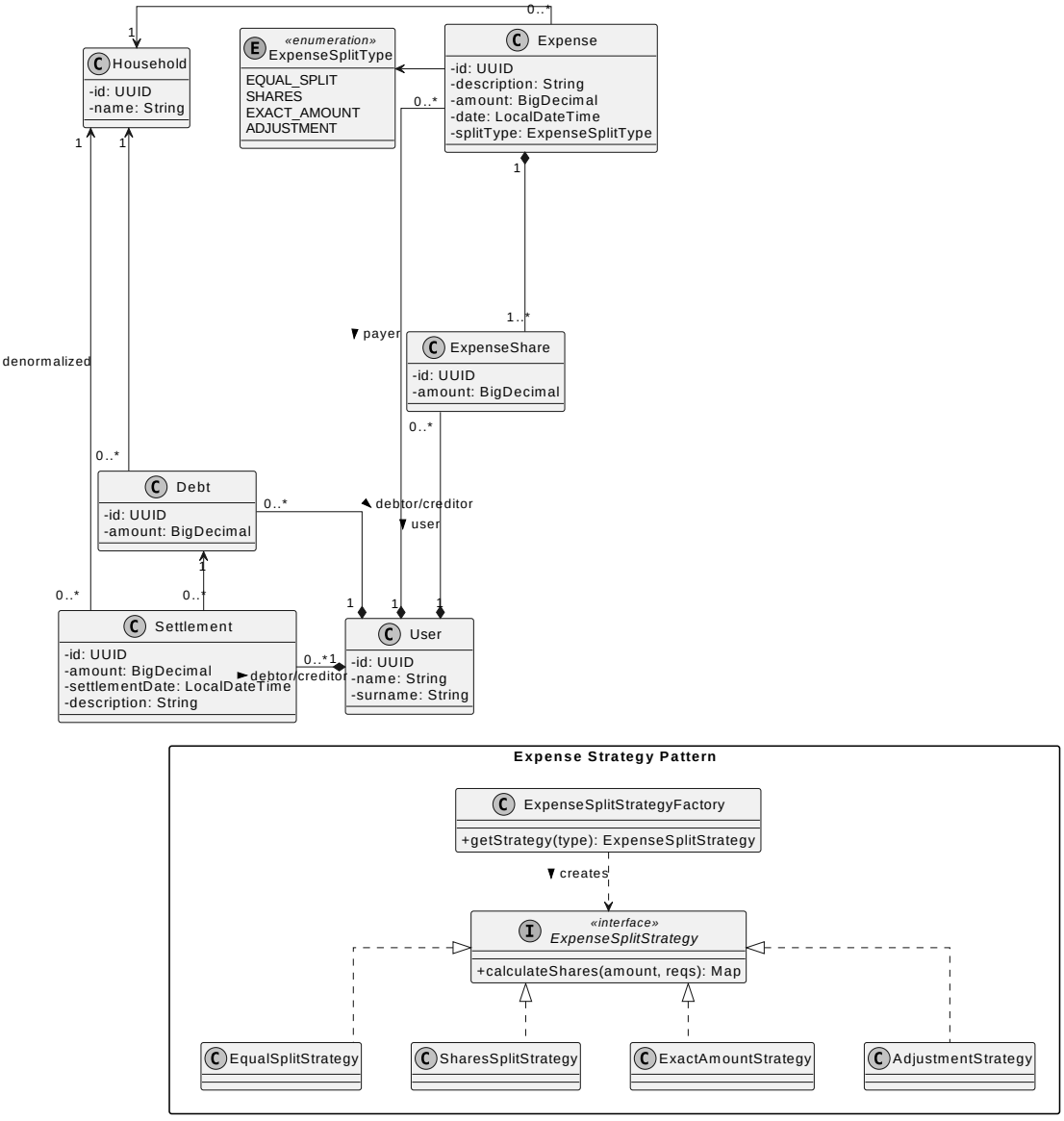

Figure 6.2: Expense Domain UML Class Diagram

6.1.4 Chore Domain Class Diagram

This diagram details the Chore Domain: chore templates, their assignments to users, and the associated status enumeration.

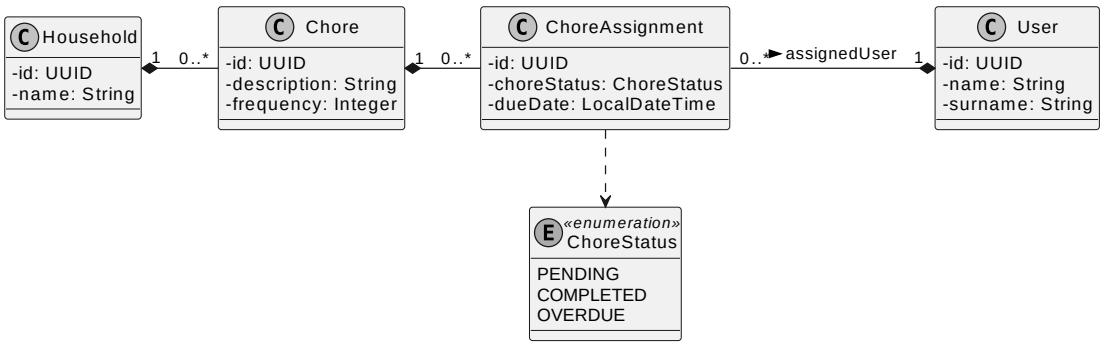

Figure 6.3: Chore Domain UML Class Diagram

6.1.5 Shopping List Domain Class Diagram

This diagram details the Shopping List Domain: shopping lists, their embedded JSON-persisted items, and the status enumeration.

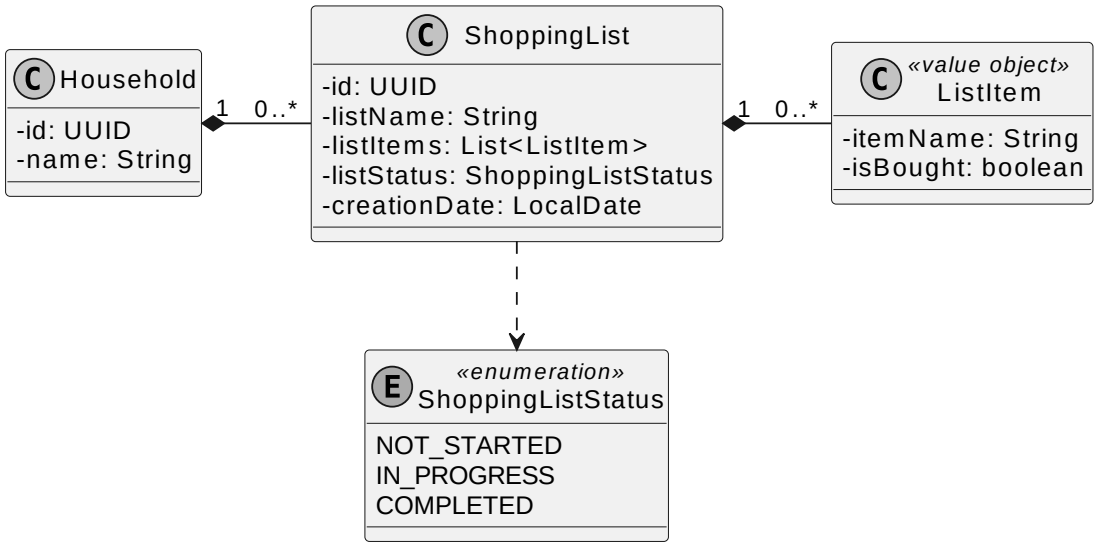

Figure 6.4: Shopping List Domain UML Class Diagram

6.1.6 Authentication and Security Class Diagram

This diagram shows the security subsystem: the JWT-based authentication filter, the service layer responsible for token management and credential verification, and the Spring Security configuration.

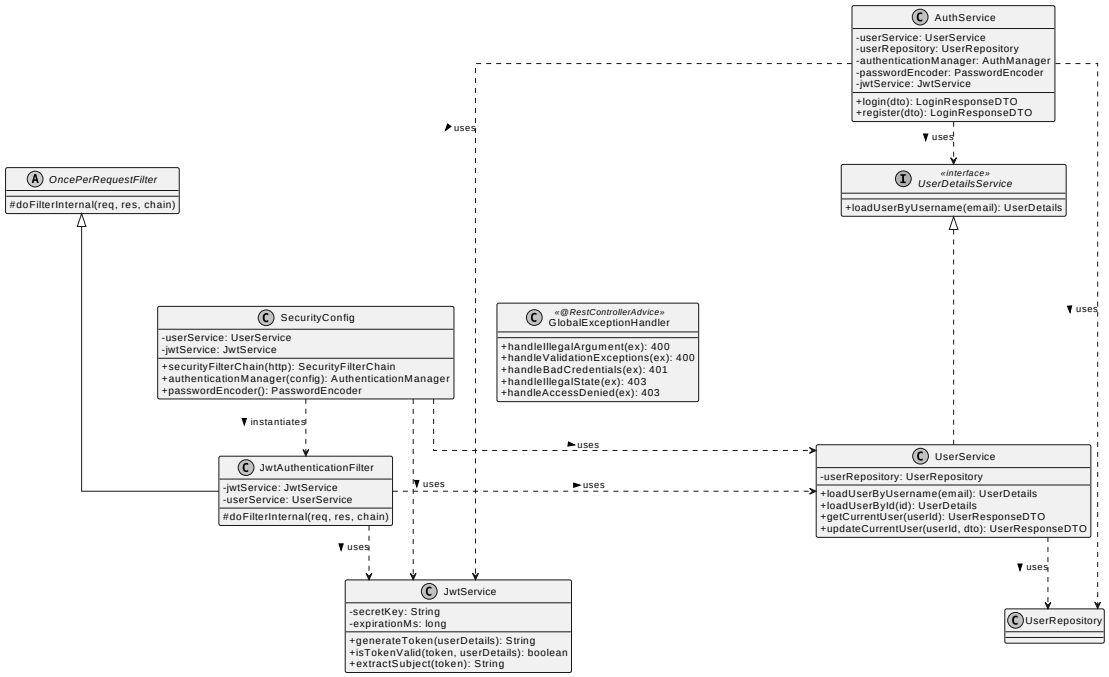

Figure 6.5: Authentication and Security UML Class Diagram

6.1.7 Service Layer Class Diagram

This diagram provides an overview of the service layer, showing how each controller delegates to its corresponding service.

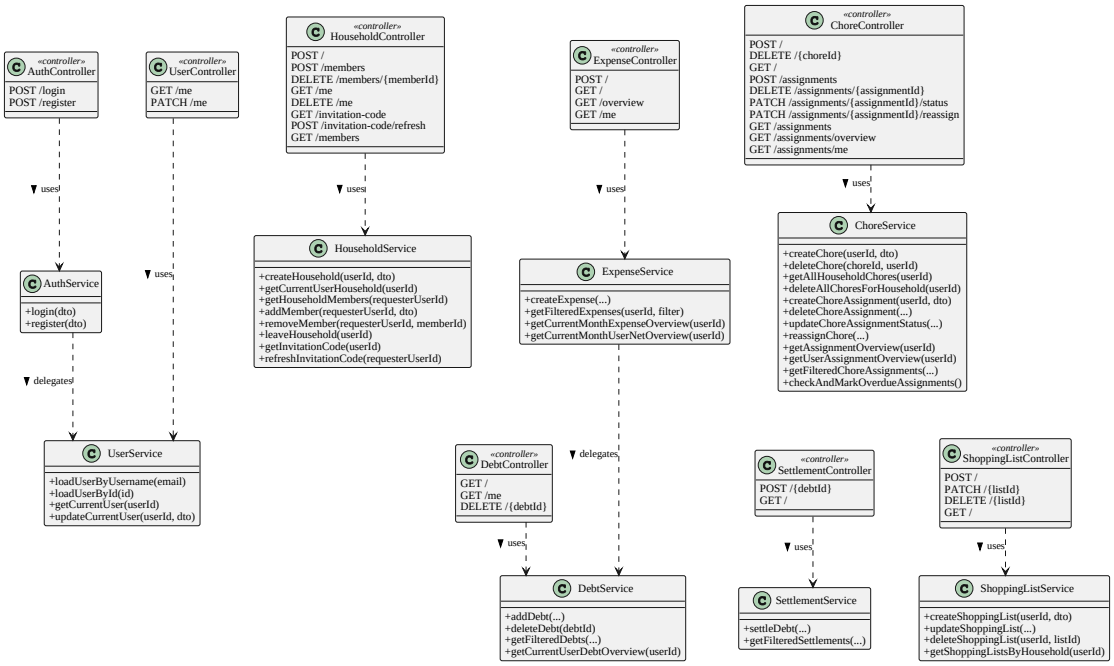

Figure 6.6: Controller and Service Layer UML Class Diagram

6.2 Domain Model Design

The backend relies on a relational database schema to model the domain of shared households. Each entity is mapped to a database table via JPA annotations, and the relationships between them are expressed through standard relational constructs (foreign keys, join tables).

6.2.1 Core Entities

The core system entities, residing in the `model` package, are described below:

- **User:** Represents an individual registered in the application. Stores profile information (name, surname, email, IBAN, payment link), hashed credentials, and a one-to-one reference to the user's current `HouseholdMembership`. A User also owns a one-to-many collection of `Unavailability` periods.
- **Household:** The central grouping entity. A Household has a name, a creation date, and a self-generated unique invitation code that external users can use

to join. It owns a one-to-many collection of `HouseholdMembership` records, each connecting a User to the Household with an admin flag and a join date.

- **Expense:** Represents a shared financial transaction logged within a household. Stores the total amount, a textual description, the payer reference, the household it belongs to, and the split type used to divide the cost. The timestamp is set automatically at creation time. An Expense owns a one-to-many collection of `ExpenseShare` children with `CascadeType.ALL` and `orphanRemoval = true`, so persisting the parent automatically persists all of its shares in a single transactional write.
- **ExpenseShare:** Represents a single user's computed portion of an Expense. Each record links back to the parent Expense and to the involved User, and stores the monetary amount that user owes. Share amounts are always strictly positive; users with a zero contribution are excluded from the share list entirely.
- **Settlement:** Records a real-world payment made by a debtor to a creditor to reduce an outstanding debt. Each Settlement references the Debt it applies to, the two involved users, the amount paid, an optional description, and a timestamp. The household identifier is denormalized from the parent Debt into the Settlement entity to enable efficient household-scoped queries without requiring a join through the Debt table.
- **Debt:** Models a directed financial obligation between two users within a household: one debtor who owes money and one creditor who is owed. Its amount can be increased by new expenses and decreased by settlements or by the automatic netting logic. Crucially, a fully settled Debt is never deleted: its amount is set to zero and the record is preserved (see Section 6.6.3).
- **Chore:** Represents a reusable household task definition. Stores a textual description and a frequency (in days) indicating how often the task should recur. A Chore belongs to exactly one Household and owns a one-to-many collection of `ChoreAssignment` children with `CascadeType.ALL` and `orphanRemoval = true`, so deleting a Chore automatically cascades to all of its assignments.
- **ChoreAssignment:** Links a specific `Chore` to a specific `User` with a due date and a status. The status is modeled as the `ChoreStatus` enumeration (`PENDING`, `COMPLETED`, `OVERDUE`). New assignments are initialized with `PENDING`; the transition to `OVERDUE` is managed by a scheduled background task (see Section 6.7.2).

- **ShoppingList:** Represents a shared shopping list within a household. Stores a name, a creation date, and an overall status (`NOT_STARTED`, `IN_PROGRESS`, `COMPLETED`). The individual items are persisted as a JSON column (via Hibernate's `@JdbcTypeCode(JSON)`) containing a list of `ListItem` objects, each with a name and a bought flag.
- **Unavailability:** Records a date range during which a User is unavailable (e.g., for chore assignment scheduling). Each record stores a start date, an end date, and a reference to its owning User.

The relationships between the described entities are clearly explicited in the following database ER diagram.

6.2.2 Complete Database ER Diagram

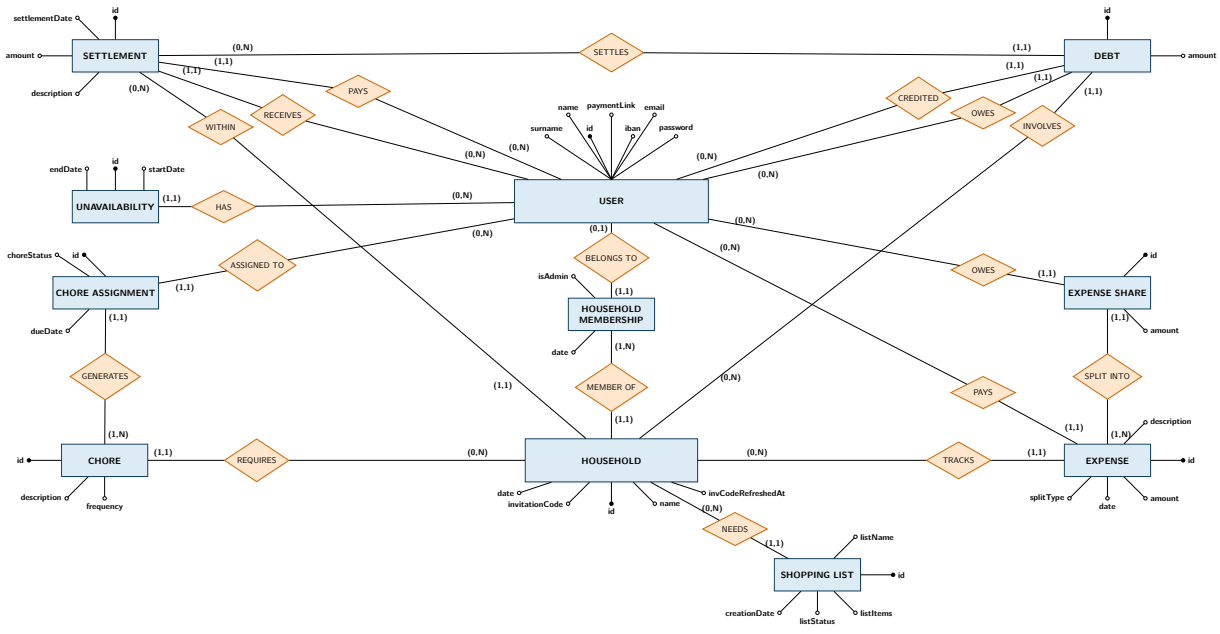

Figure 6.7: Database ER Diagram

6.3 Security Implementation

The application's security architecture is built on Spring Security and relies on JSON Web Tokens (JWTs) for stateless authentication.

6.3.1 Architectural Decisions and Consequences

The security policies that dictate how the application handles state and authentication are defined in `SecurityConfig`:

- **Stateless Sessions:** The session management policy is strictly set to `STATELESS`, meaning the server does not allocate memory for HTTP sessions (`JSESSIONID`), which would require either sticky sessions or distributed session caches once the server becomes distributed across multiple machines. This guarantees that the backend can scale horizontally without synchronization overhead, as every request must independently provide its credentials via the JWT.
- **Token-Based Authentication:** Upon successful authentication at the public `/api/auth/**` endpoints, the system issues to the user a JWT signed with an HMAC-SHA256 secret. The token's `subject` claim embeds the user's UUID, avoiding querying the database to verify a token's integrity. Cryptographic signature validation alone guarantees that the token was issued by the backend and has not been tampered with.
- **Password Hashing:** Credentials are hashed using Spring's `BCrypt` password encoder, which inherently incorporates a random salt generator and a configurable work factor. This decision obviously avoids raw passwords to be saved, but also protects the database against rainbow-table attacks and slows down brute-force attempts. This increase in robustness makes the cost of slightly higher CPU usage during the login process negligible.

6.3.2 The JWT Authentication Filter

The `JwtAuthenticationFilter` extends Spring's `OncePerRequestFilter` to intercept every incoming HTTP request and block the ones that are trying to access protected endpoints without a valid JWT. Its processing flow is presented in the following simplified code snippet (the actual code contains JWT and user ID sanity checks):

```java
public class JwtAuthenticationFilter extends OncePerRequestFilter {
    @Override
    protected void doFilterInternal(
        HttpServletRequest request,
        HttpServletResponse response,
        FilterChain filterChain
    ) {
        final String authHeader = request.getHeader("Authorization");
        if (authHeader == null || !authHeader.startsWith("Bearer ")) {
            filterChain.doFilter(request, response);
            return;
        }

        final String jwt = authHeader.substring(7);
        final String userIdString = jwtService.extractSubject(jwt);

        if (
            userIdString != null &&
            SecurityContextHolder.getContext().getAuthentication() == null
        ) {
            final UUID userId = UUID.fromString(userIdString);
            UserDetails userDetails = userService.loadUserById(userId);

            if (jwtService.isTokenValid(jwt, userDetails)) {
                UsernamePasswordAuthenticationToken authToken = new
                    UsernamePasswordAuthenticationToken(
                        userDetails, null, userDetails.getAuthorities()
                );
                SecurityContextHolder.getContext().setAuthentication(authToken);
            }
        }
        filterChain.doFilter(request, response);
    }
}
```

Listing 6.1: JWT Validation Lifecycle (Simplified)

The filter retrieves the JWT from the `Authorization` header of the request. After extracting the user ID from the token payload, it loads the corresponding `UserDetails` from the database, cryptographically validates the token signature and expiration, and then populates Spring's `SecurityContext`. This allows downstream controllers to retrieve the authenticated user via the `@AuthenticationPrincipal` annotation. If any step fails (missing header or token, invalid UUID format, expired token, unknown user), the filter passes the request to the next filter chain component without authentication.

The `JwtAuthenticationFilter` does not block the requests autonomously but delegates this decision to Spring Security, which will accept unauthenticated requests to non protected endpoints but return a `401 Unauthorized` response for protected ones.

6.4 Global Exception Management

The backend employs a centralized exception handling mechanism via the `GlobalExceptionHandler` class. Annotated with `@RestControllerAdvice`, this interceptor catches exceptions thrown by any controller and translates them into HTTP responses with appropriate status codes, preventing raw stack traces from leaking to the client. The following mappings are defined:

- `IllegalArgumentException` → **400 Bad Request**: returned when a required parameter is missing or semantically invalid.
- `BindException` → **400 Bad Request**: returned when Jakarta Bean Validation annotations (e.g. `@NotBlank`, `@Email`) fail. The response body contains a JSON map of field names to their respective error messages.
- `BadCredentialsException` → **401 Unauthorized**: returned when login credentials are invalid.
- `IllegalStateException` → **403 Forbidden**: returned when an operation violates a business rule (e.g. a user attempting to create a household while already belonging to one).
- `AccessDeniedException` → **403 Forbidden**: returned when a non-admin user attempts an admin-only operation, or when a user tries to access a resource outside their household.

6.5 Core Business Logic: Household Management

The household subsystem is the foundational pillar of the application. Every shared resource (expenses, chores, shopping lists) is tightly scoped to a specific `Household`.

6.5.1 Role-Based Access Control (RBAC)

In order to ensure a role-based access control to the household content the system distinguishes between standard members and administrators at the household level. Administrators have elevated privileges that allow them to refresh the household invitation code (invalidating the previous one), remove other members from the household and perform destructive operations on shared resources (e.g. deleting a

chore with all its assignments). These rules are enforced via programmatic checks in the service layer before making any persistent change.

6.5.2 Household Management Flow

When a user decides to create a new household, the `HouseholdService` executes the following steps transactionally:

1. Validates that the requesting user does not already belong to another household, ensuring a strict one-to-one mapping between a user and their active home.
2. A unique invitation code is securely generated and assigned to the new household.
3. A `HouseholdMembership` entity is instantiated to link the user to the household, with the `isAdmin` flag explicitly set to `true`.

Other users can join the household by submitting the active invitation code. The system verifies the code, ensures the joining user is not already part of a household and creates a new `HouseholdMembership` with the `isAdmin` flag set to `false`.

6.6 Core Business Logic: Expense Management

One of the most important components of HouseMate is its financial subsystem. It comprehends expense recording, debt tracking, settlement processing and aggregated overviews computation. All database write operations are wrapped in Spring `@Transactional` boundaries to guarantee Atomicity, Consistency, Isolation and Durability (ACID properties). This way, if any step within a transaction fails, the entire operation is rolled back, preventing an inconsistent or partial state from reaching the database.

6.6.1 Expense Creation Flow

The creation of an expense is the most complex transactional operation in the system, as it must atomically perform three distinct tasks: persist the user-directed expense record itself, compute and store each user's individual share, and update all affected debt balances.

In order to create an expense, the following steps execute within a single database transaction:

1. The requesting user (the payer) and their current household entities are fetched from the database. If the user is not a member of any household the operation is rejected immediately.
2. A new `Expense` entity is initialized with the provided description, total amount, payer reference, household and split type.
3. The appropriate split calculation strategy is obtained from the `ExpenseSplitStrategyFactory` (discussed in Section 6.6.2) and invoked to produce a map of the participants IDs to evaluated share amounts.
4. For each entry in the computed map, an `ExpenseShare` child entity is created and added to the parent `Expense`'s collection.
5. The resulting graph, composed by the new expense and its shares, is persisted via a unique call to the `ExpenseRepository save` method, thanks to JPA's `CascadeType.ALL` annotation on the relationship between the two entities. At the same time, `Spring`'s `@Transactional` annotation ensures the graph is saved in a single database unit of work, achieving the intended behaviour.
6. For every share whose user differs from the payer, the `DebtService addDebt` method is called to update their debt register to account for the new debt owed to the payer (discussed in Section 6.6.3).

If any step fails (e.g. a participant ID does not exist, or the split strategy detects an invalid configuration) the `@Transactional` annotation ensures Spring Data immediately rolls back the entire operation, preventing "ghost" expenses or orphaned share records from corrupting the database.

6.6.2 The Strategy Pattern for Expense Splitting

The computation of individual shares is decoupled from the expense creation logic through the Strategy design pattern. The `ExpenseService` acts as the *Context* and delegates the share computation to the `ExpenseSplitStrategy` interface, which is implemented by four concrete strategies.

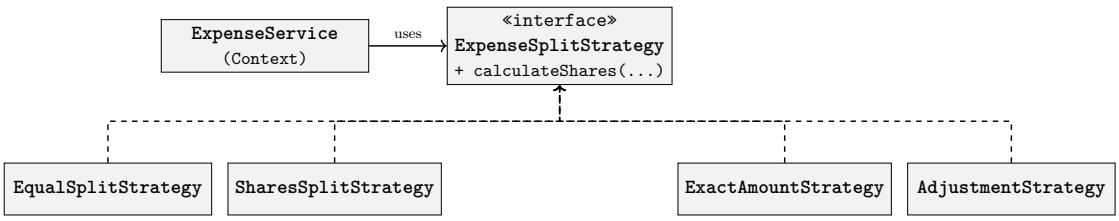

Figure 6.8: Strategy Pattern: Expense Split Strategies

The `ExpenseSplitStrategy` interface declares the contract that all splitting algorithms must satisfy:

```
public interface ExpenseSplitStrategy {
    Map<UUID, BigDecimal> calculateShares(
            BigDecimal amount,
            List<ExpenseShareRequestDTO> shareRequests);
}
```

Listing 6.2: The ExpenseSplitStrategy Interface

All monetary values throughout the expense subsystem are represented using `BigDecimal` rather than primitive floating-point types such as `double` or `float`. This is a deliberate choice motivated by the fact that IEEE 754 binary floating-point arithmetic cannot exactly represent common decimal fractions (e.g. `0.1`), leading to silent rounding errors that accumulate over repeated additions and divisions, which come up to be not neglectable. `BigDecimal` provides arbitrary-precision decimal arithmetic and all entity columns are mapped with `precision=10` and `scale=2` to enforce two-decimal-place storage at the database level. A direct consequence of this choice is that division operations such as splitting $10.00 among three users can produce non-terminating decimals; the cent-distribution algorithm described below exists precisely to handle this scenario in a mathematically exact way.

Each of the four concrete `ExpenseSplitStrategy` implementations is mapped to the corresponding value of the `ExpenseSplitType` enumeration:

- **`EQUAL_SPLIT`:** Divides the total amount equally among all involved users. The base share is computed by truncating the division result to two decimal places (using `RoundingMode.DOWN`) and the resulting remainder is distributed as one-cent increments in a round-robin fashion across the participants. This guarantees that the sum of all shares equals the original total exactly, without creating or destroying any cents. Any minor discrepancies where some users

receive one cent more than others are acceptable trade-offs for ensuring total financial accuracy.

- **`SHARES`:** Proportionally splits the total, according to abstract numeric weights provided by the payer. Each user's share is calculated by truncating to two decimal places the result of:

$$\frac{\texttt{weight}}{\texttt{totalWeights}} * \texttt{totalAmount}$$

  A second pass distributes the remaining cents using the same rotation technique used for `EQUAL_SPLIT`.

- **`EXACT_AMOUNT`:** Each user's share is provided directly as an exact monetary value by the payer. The strategy validates that all individual amounts are non-negative and that their sum matches the total expense amount exactly. If a household member is selected as a participant but their share amount is set to 0 it gets filtered out to keep the database clean.

- **`ADJUSTMENT`:** This hybrid approach begins with an equal baseline split but allows the payer to apply positive or negative adjustments to specific users (e.g. specifying that a participant owes $20 more than the others, while another owes $50 less). The algorithm first subtracts the sum of all adjustments from the total amount and then equally divides the remaining balance with cents distribution. Each user's final share gets computed by adding their individual adjustment to this baseline portion. For financial integrity and database cleanness reasons, the strategy rejects any configuration that results in a user having a negative final share and filters out shares that evaluate to exactly zero.

Comparison with the standard GoF Strategy pattern.

- **Interface vs Abstract Class:** The standard GoF Strategy pattern allows the Strategy to be either an interface or an abstract class. Our implementation uses a Java interface, which is the preferred choice in modern Java because it avoids locking the concrete strategies into a single inheritance chain, leaving them free to extend other classes if necessary.

- **Context–Strategy coupling:** In the classic pattern, the Context holds a direct reference to the current Strategy and delegates to it.
  In our implementation, the `ExpenseService` does not retain a persistent strategy reference; instead, it obtains a fresh strategy instance from `ExpenseSplitStrategyFactory` on every invocation (see next paragraph).

This is a deliberate choice enabled by Spring's singleton bean lifecycle: all strategies are stateless, so there is no need to cache or swap them.

The Simple Factory Pattern for Strategy Resolution

The resolution of the appropriate strategy is delegated to the class `ExpenseSplitStrategyFactory`. This Spring `@Component` receives all four concrete strategy beans via constructor injection; its single public method, `getStrategy`, maps each `ExpenseSplitType` enumeration value to its corresponding Spring-managed bean through a `switch` expression:

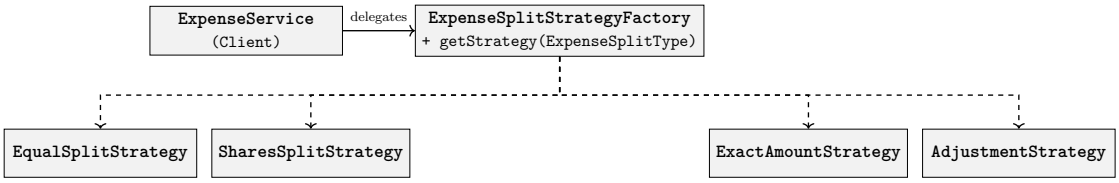

Figure 6.9: Simple Factory Pattern: Strategy Resolution

```
public ExpenseSplitStrategy getStrategy(ExpenseSplitType splitType
    ) {
    return switch (splitType) {
        case EQUAL_SPLIT  -> equalSplitStrategy;
        case SHARES       -> sharesSplitStrategy;
        case EXACT_AMOUNT -> exactAmountStrategy;
        case ADJUSTMENT   -> adjustmentStrategy;
    };
}
```

Listing 6.3: Expense Split Strategy Factory switch expression

This centralizes the strategy-resolution logic in one place: the `ExpenseService` never needs to know which concrete class implements a given split type.

Comparison with the standard GoF Factory pattern.

- **Simple Factory vs Factory Method:** The implementation does not follow the classic *Factory Method* pattern, in which the creator is an abstraction and instantiation is delegated to subclasses. Instead, it follows the *Simple Factory* idiom: a single concrete class uses conditional logic (the `switch` expression) to select among existing instances. This is a pragmatic choice appropriate to the scale of the problem: a full Factory Method hierarchy would introduce

unnecessary abstraction layers without a tangible benefit, since the set of split types is closed (defined by an enumeration) and unlikely to grow frequently.

- **Instance reuse vs creation:** In the classic Factory pattern, the factory *creates* new objects on each call. Our factory instead *returns* pre-existing singleton beans managed by Spring's IoC container. Because the strategies are stateless, this reuse is safe and eliminates the overhead of repeated object construction.

6.6.3 Debt Lifecycle and Netting Algorithm

Debts are never created or managed directly by the user, they instead are a derived consequence of expenses and settlements. The `DebtService addDebt` method implements a netting algorithm that maintains a compact and consistent debt situation across the household.

When a new debt of value d is to be recorded from user A (debtor) to user B (creditor) within a household, the algorithm proceeds as follows:

1. If A and B are the same user, the operation is a no-op (a user cannot owe themselves).
2. The system checks whether an *inverse* debt exists, that is to say a record where B owes A in the same household. If such a record exists:
   - If the inverse debt's amount is greater than d, the inverse debt is reduced by d and the operation terminates. No forward debt is created.
   - If the inverse debt's amount equals d, the two obligations cancel out perfectly. The inverse debt's amount is set to zero and the operation terminates.
   - If the inverse debt's amount is less than d, the inverse debt is zeroed out and the algorithm continues with the remaining difference as the new d.
3. After netting, the system checks whether a forward debt already exists from A to B. If so, the existing amount is incremented by d. Otherwise, a new `Debt` entity is created with amount d.

The following listing shows the core of this algorithm as implemented in the `DebtService`:

```
public void addDebt(
    UUID debtorId, UUID creditorId, UUID householdId, BigDecimal amount
) {
```

```
4       if (debtorId.equals(creditorId)) return;
5
6       BigDecimal netAmount = amount;
7       final Debt inverseDebt = debtRepository
8           .findByDebtorAndCreditorAndHousehold(creditor, debtor, household)
9           .orElse(null);
10
11      if (inverseDebt != null) {
12          final int cmp = inverseDebt.getAmount().compareTo(amount);
13          if (cmp > 0) {
14              inverseDebt.setAmount(inverseDebt.getAmount().subtract(amount));
15              debtRepository.save(inverseDebt);
16              return;
17          } else if (cmp == 0) {
18              inverseDebt.setAmount(BigDecimal.ZERO);
19              debtRepository.save(inverseDebt);
20              return;
21          } else {
22              netAmount = amount.subtract(inverseDebt.getAmount());
23              inverseDebt.setAmount(BigDecimal.ZERO);
24              debtRepository.save(inverseDebt);
25          }
26      }
27
28      final Debt existingDebt = debtRepository
29          .findByDebtorAndCreditorAndHousehold(debtor, creditor, household)
30          .orElse(null);
31      if (existingDebt != null) {
32          existingDebt.setAmount(existingDebt.getAmount().add(netAmount));
33          debtRepository.save(existingDebt);
34      } else {
35          debtRepository.save(new Debt(debtor, creditor, household, netAmount));
36      }
37  }
```

Listing 6.4: Debt Netting Algorithm (Simplified)

It is a design decision to *not* delete from the database any debt record even if its amount reaches zero, whether through settlement, netting, or exact cancellation. Instead, the row is preserved with a zero balance to avoid a wasteful cycle of destroying and recreating structurally identical entities (same debtor, same creditor, same household) whenever financial balances fluctuate due to frequent new expenses and settlements; the zero-amount row acts as a reusable slot to be incremented when a new debt between the two members is originated.

This approach also guarantees a key invariant: for any ordered pair of users within a household, at most one debt record exists at any given time, regardless of how many expenses they have been involved into.

6.6.4 Settlement Processing

A settlement represents a real-world payment made by a debtor to a creditor. The `SettlementService settleDebt` method executes the following steps within a single transaction:

1. The target `Debt` entity and the requesting user are fetched from the database.
2. The system validates that the requesting user is the debtor on the referenced debt, that the specified creditor matches the debt record and that the settlement amount is strictly positive and does not exceed the current debt balance.
3. A new `Settlement` entity is created to record the debt reference, both parties, the amount, the current timestamp and an optional description. The household is denormalized from the debt into the settlement for query efficiency; since both the debt and the settlement must always be scoped to the same household and cannot be moved to a different one, this denormalization does not induce data anomalies.
4. The debt's balance is reduced by the settlement amount. If the settlement fully covers the debt, the balance is left to zero consistently with the persistence policy described above.

6.6.5 Dynamic Query Filtering with the Specification Pattern

All list-retrieval operations across the financial (and chore) subsystems support dynamic, composable filtering through a Specification pattern, implemented via the *JPA Criteria API*.

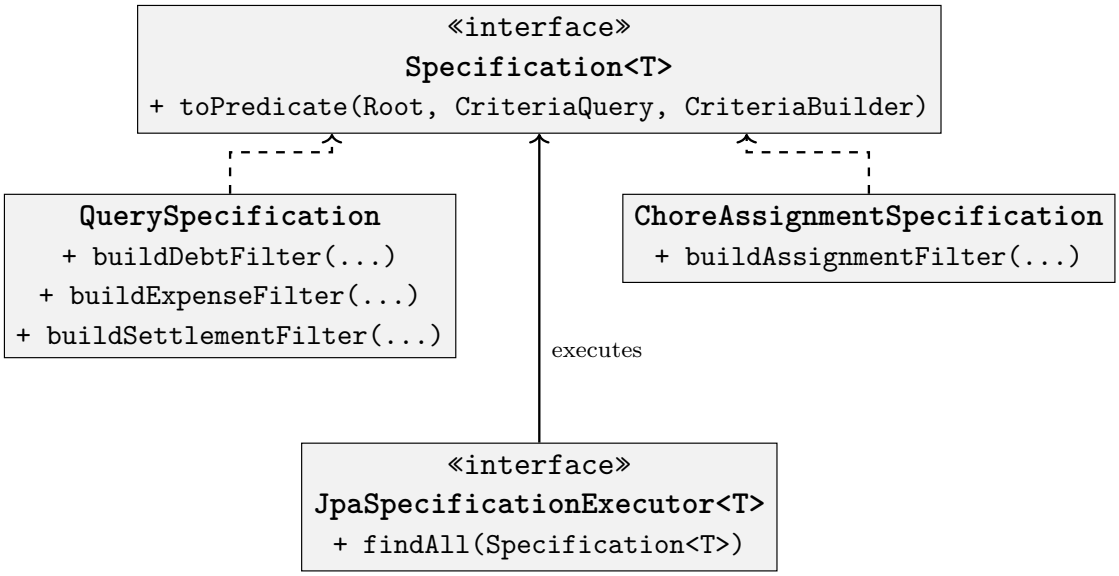

Figure 6.10: Specification Pattern: Dynamic Query Construction

Implementation description. Two utility classes, `QuerySpecification` (for expenses, debts and settlements) and `ChoreAssignmentSpecification` (for chore assignments), provide static factory methods that convert filter DTOs into `Specification` lambda expressions. Each method builds a list of `Predicate` objects from the non-null fields in the filter DTO, combining them with a logical AND. The resulting specification is then passed to the repository's `findAll(Specification)` method for execution. This approach dynamically builds the database queries based on the user filtering requests, avoiding the need to write a combinatorial amount of them beforehand; in fact, without this pattern, each combination of optional filters would require a separate named query method in the repository (e.g. to filter debts we would need `findByHouseholdAndDebtor`, `findByHouseholdAndCreditor`, `findByHouseholdAndDebtorAndDateRange`, etc.), leading to a combinatorial explosion. The Specification pattern reduces this to a single `findAll(Specification)` call, regardless of how many filters are active.

Comparison with the standard Specification pattern.

- **Domain-Driven Design:** The original Specification pattern encapsulates a business rule as a composable, reusable object. Spring Data JPA's `Specification` interface is a direct implementation of this concept, mapping the `isSatisfiedBy` method to the *JPA Criteria API*'s `toPredicate` method.

- **Composability:** The standard pattern emphasizes composing specifications via logical operators (`and`, `or`, `not`). Our implementation achieves this internally by building a list of predicates and combining them with `criteriaBuilder.and()`, which is functionally equivalent. The static factory method approach was chosen over individual specification classes because the filter combinations are always conjunctive (AND-only) and do not require the full expressive power of arbitrary composition.

6.7 Core Business Logic: Chore Management

HouseMate's chore subsystem handles the lifecycle of household tasks: defining reusable chore templates, assigning them to individual members with due dates, tracking completion status and automatically detecting overdue assignments.

6.7.1 Chore Assignment Lifecycle

When a new chore assignment is created through the `ChoreService`, the following validations and steps are executed within a single transaction:

1. The system verifies that the due date is at least one hour in the future, rejecting assignments with immediate or past deadlines.
2. The referenced `Chore` and the target `User` are resolved from the database and the system verifies that both the requesting user and the target user belong to the chore household.
3. A new `ChoreAssignment` entity is created with an initial status of `PENDING`.
4. Only the user to whom the assignment is addressed can update its status (e.g. marking it as `COMPLETED`). Any request from another user attempting this operation will be rejected.

Only the household administrators can delete a chore and its assignments. Moreover, when deleting a chore template, all of its assignments get deleted as well, thanks to *JPA*'s `CascadeType.ALL` annotation on the relationship between these two entities.

6.7.2 Scheduled Overdue Detection

A background scheduled task runs every five minutes (configured via the `@Scheduled` cron expression `"0 0/5 * * * ?"`) to automatically transition overdue assignments.

The task issues a single bulk-update query that sets the status of all `PENDING` assignments whose due date has passed to `OVERDUE` simultaneously. This approach avoids loading entities into memory and operates directly at the database level for efficiency.

```
@Scheduled(cron = "0 0/5 * * * ?")
@Transactional
public void checkAndMarkOverdueAssignments() {
    int updatedCount = choreAssignmentRepository
        .markOverdueAssignments(LocalDateTime.now());
}
```

Listing 6.5: Scheduled Overdue Detection

6.8 Core Business Logic: Shopping List Management

The shopping list subsystem allows household members to collaboratively track groceries and shared supplies.

6.8.1 List Lifecycle and JSON Persistence

Each `ShoppingList` belongs to a household and transitions through three states: `NOT_STARTED`, `IN_PROGRESS`, `COMPLETED`.

Rather than creating a separate database table and entity for individual list items, they are embedded directly within the `ShoppingList` entity as a JSON document:

```
@Column(name = "items", nullable = false)
@JdbcTypeCode(org.hibernate.type.SqlTypes.JSON)
private List<ListItem> listItems;
```

Listing 6.6: JSON Mapping for Shopping List Items

This architectural decision has been made not only to optimize read/write performance (since shopping list items are always retrieved and updated in bulk alongside their parent list, avoiding a one-to-many SQL join significantly reduces database overhead), but also because, considering each item can belong to only one shopping list, this means that for just two lists with similar items the database would realistically have an items table with many almost duplicated records.

6.8.2 Status Derivation

Whenever a member checks or unchecks an item on the list, the `ShoppingListService` dynamically re-evaluates the overall status of the list:

- If none of the items are marked as bought, the status is `NOT_STARTED`.
- If all of the items are marked as bought, the status is `COMPLETED`.
- Otherwise, the status is `IN_PROGRESS`.

7. Frontend Design and Implementation

7.1 Client Module UML Diagram

In the same way as the backend module, the detailed description of the client module begins with the complete UML Class Diagrams of each individual domain area of the application.

7.1.1 Expense Domain Class Diagram

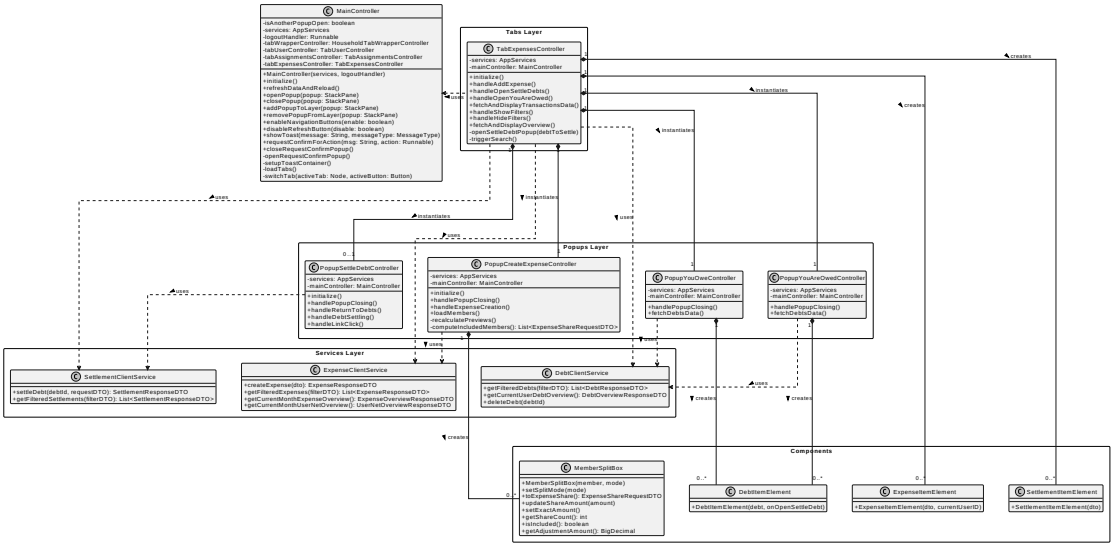

Figure 7.1: Client Module Expense Domain UML Class Diagram

7.1.2 Chore Domain Class Diagram

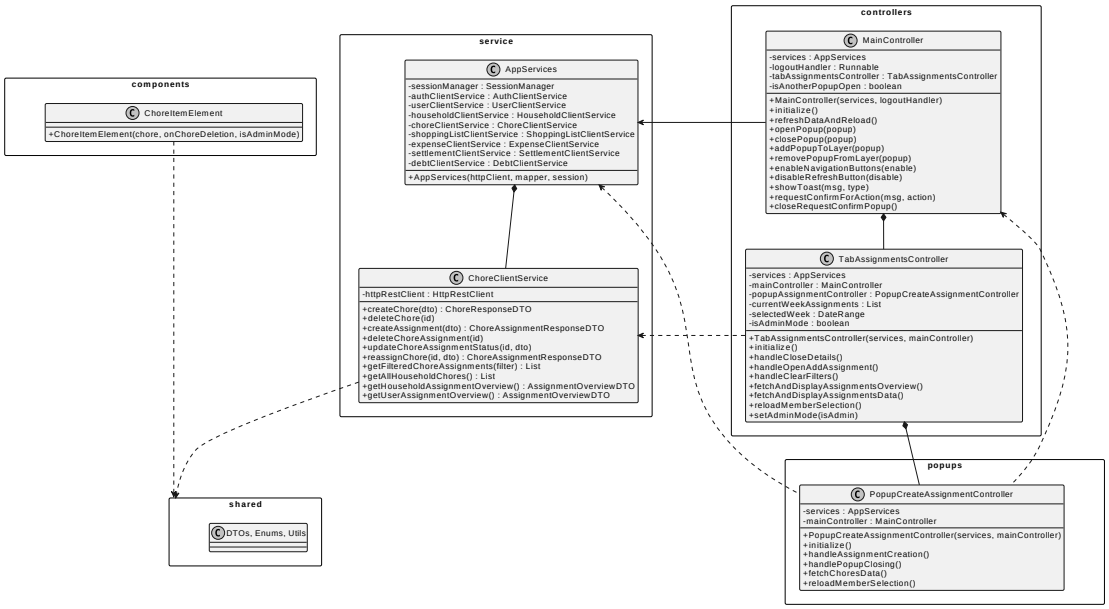

Figure 7.2: Client Module Chore Domain UML Class Diagram

7.1.3 Household Domain Class Diagram

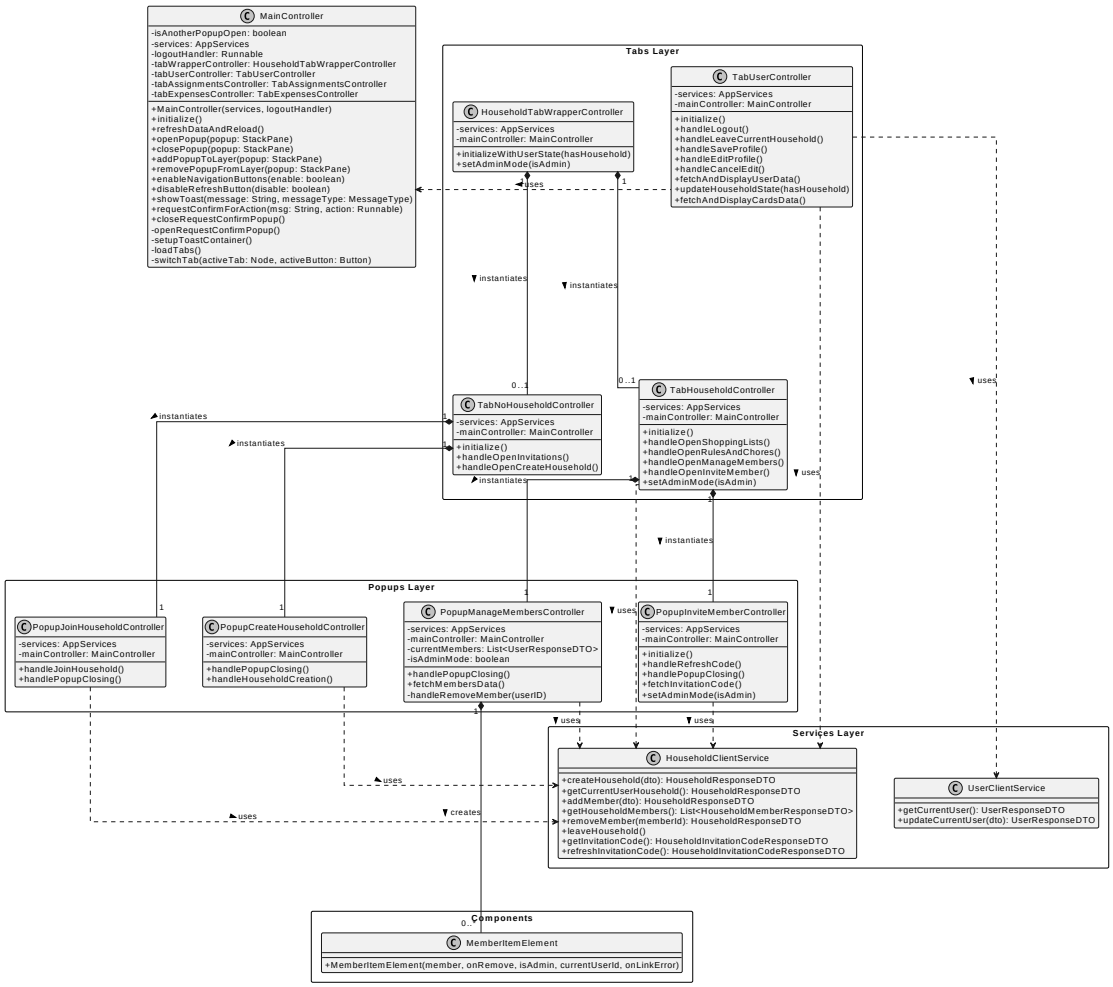

Figure 7.3: Client Module User-Household Domain UML Class Diagram

7.2 Presentation Layer Design

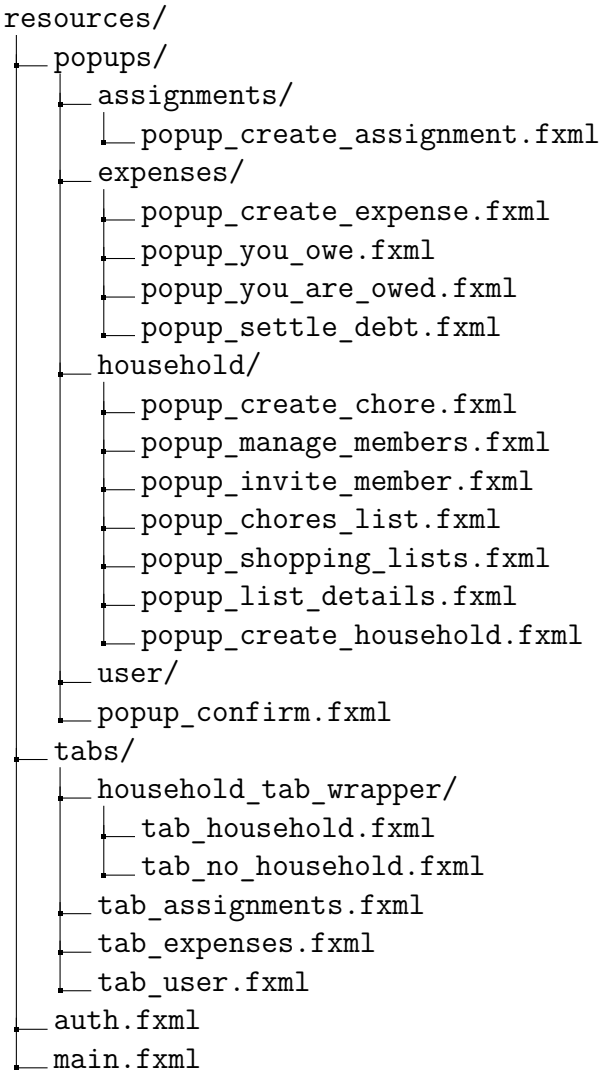

Figure 7.4: Frontend FXML Structure

The application's user interface is built on a single main window that displays content only after the user successfully authenticates. The entire interface follows a hierarchical structure: the main screen contains all sub-components specific to each functionality, as the directory tree illustrates. Each static UI component is defined in its own FXML file.

The authentication screen (`auth.fxml`) is completely separated from the main application area. The `main.fxml` file contains all remaining sub-components, which are loaded into a dedicated container node. The user interface makes use of popups to display and allow interaction with the application's main functionalities.

The following screenshots, taken directly from the finished project, illustrate how the main application is composed of four tabs (Household, Assignments, Expenses, User) and how each tab opens its own popups for specific functionalities. The "Confirm Action" popup sits over the

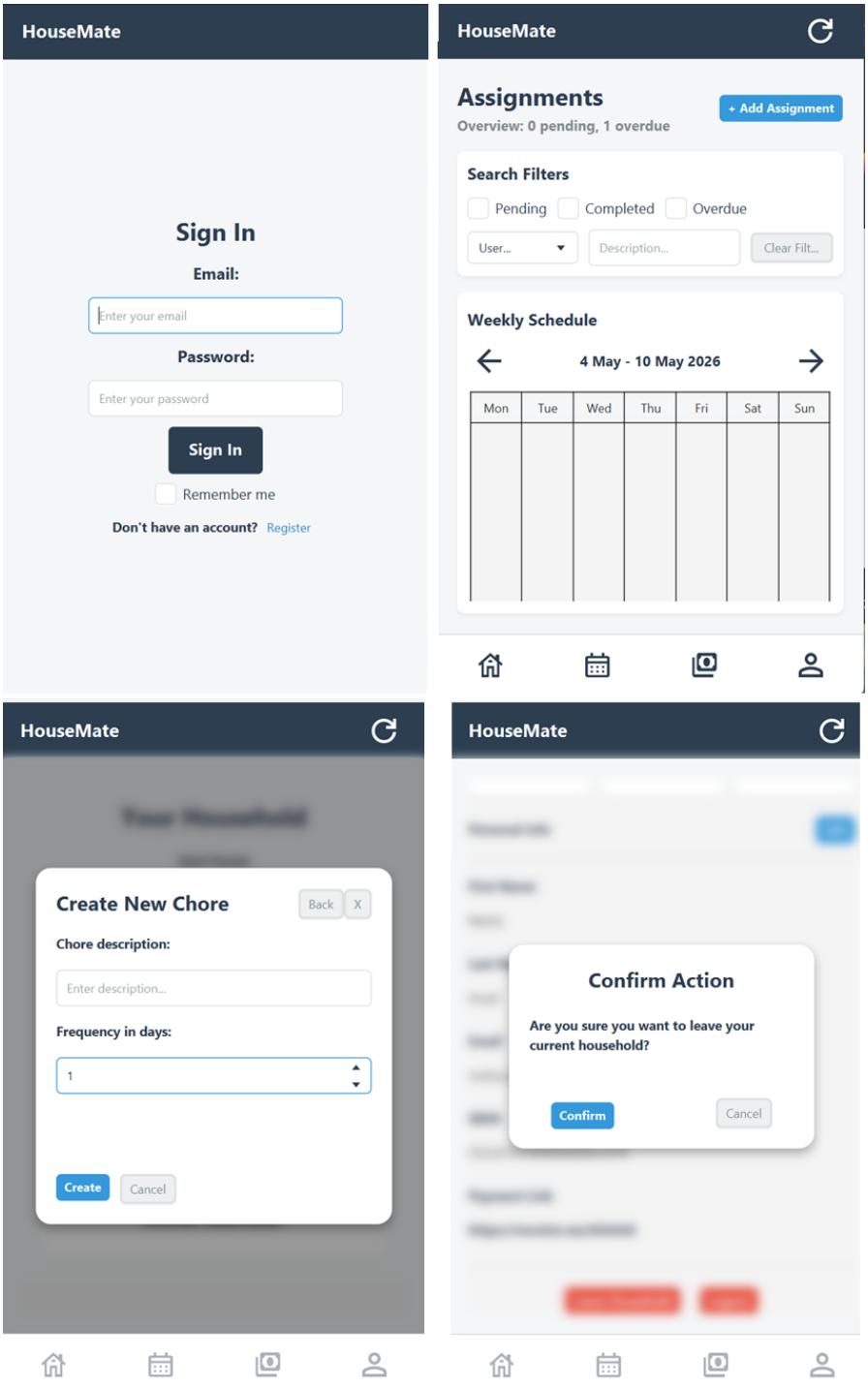

Figure 7.5: Examples of the user interface's appearance and structure

7.3 Frontend-Backend Communication

7.3.1 Performing HTTP Requests

The client module communicates with the backend by performing HTTP requests to the REST endpoints it exposes. This responsibility is assigned to the `ClientService` classes, one for each principal domain entity. These classes expose simple Java methods to the UI-handling code, acting as bridges between Java objects and the JSON payloads exchanged with the backend. All client services are composed with a shared helper class, `HttpRestClient`, which has two responsibilities: performing JSON serialization and deserialization via the `ObjectMapper` class from the Jackson library, and sending HTTP requests using the native `java.net.http.HttpClient` module. Using these helper methods, the service methods remain concise:

```
1  public UserResponseDTO updateCurrentUser(UserUpdateRequestDTO
       requestDTO) {
2      String requestBody = httpRestClient.serializeDTO(requestDTO);
3
4      HttpRequest request = HttpRequest.newBuilder()
5              .uri(URI.create(BASE_URL + "/api/users/me"))
6              .header("Content-Type", "application/json")
7              .header("Accept", "application/json")
8              .header("Authorization", httpRestClient.
       buildAuthHeader())
9          .method("PATCH", HttpRequest.BodyPublishers.ofString(
       requestBody))
10             .build();
11
12     HttpResponse<String> response = httpRestClient.sendRequest(
       request);
13
14     if (response.statusCode() == 200) {
15         return httpRestClient.deserializeDTO(response.body(),
       UserResponseDTO.class);
16     }
17
18     throw new RuntimeException(
19             "Failed to update current user. Server responded with
       status code: " + response.statusCode() + " and message: " +
       response.body()
20     );
```

```
21 }
```

Listing 7.1: Example of an HTTP request method in the client services (UserClientService)

7.3.2 Client-Side State Management

The client maintains key session data locally to avoid redundant backend calls. Three classes collaborate to implement this:

- `SessionManager`: caches response DTOs for the currently logged-in user, their household, and household members. This data is refreshed explicitly by the user and reused across multiple operations. Without it, the client would need to issue additional requests for almost every functionality, effectively doubling the load on the server.
- `JwtPersistanceHandler`: implements the "Remember Me" functionality. When the user opts in, this class stores the JWT received from the login endpoint in the system registry via the native `java.util.prefs.Preferences` API. On subsequent launches, it retrieves the stored token and attempts authentication without requiring the email and password. If the token has expired or is otherwise invalid, the application falls back to the standard login screen.
- `AppServices`: a wrapper class to simplify the injection of the dependencies through the Controller hierarchy.

The `AppServices` class acts as a Facade over the entire client-side service subsystem. It aggregates instances of all eight client services (`AuthClientService`, `UserClientService`, `HouseholdClientService`, `ChoreClientService`, `ExpenseClientService`, `DebtClientService`, `SettlementClientService`, `ShoppingListClientService`), the `SessionManager`, and the underlying `HttpRestClient`. Controllers never instantiate or manage service dependencies themselves; they receive a single `AppServices` reference during FXML loading and use its getters to access any service they need.

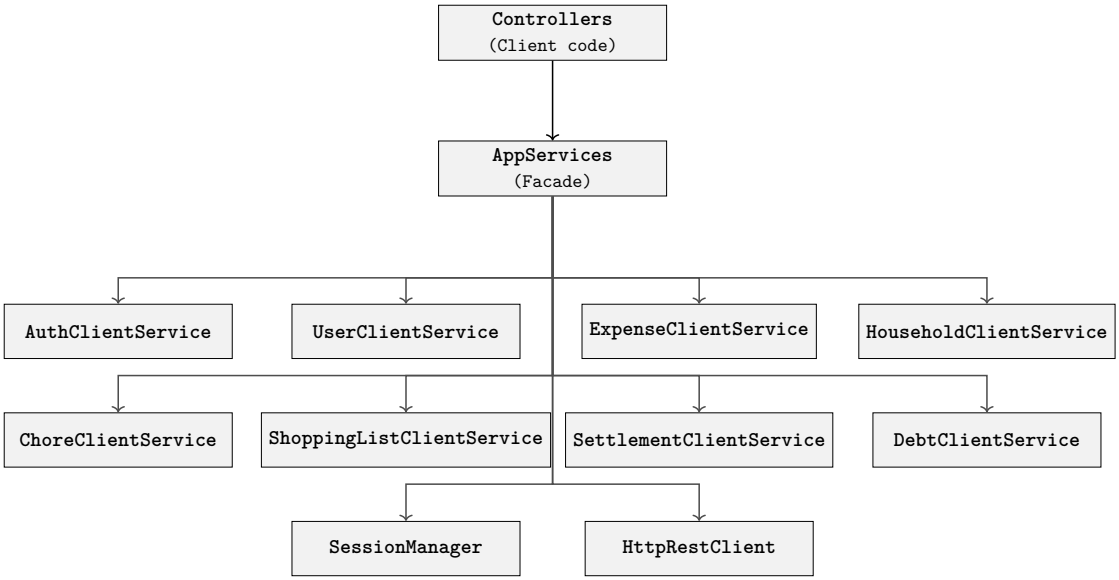

Figure 7.6: Facade Pattern: Unified Service Access

Comparison with the standard design pattern.

- **Simplified interface:** The GoF Facade pattern provides a unified, simplified interface to a complex subsystem. `AppServices` fulfills this precisely: without it, every controller would need to receive and manage references to eight independent services plus the session manager, resulting in extensive boilerplate code in each controller's constructor.
- **Not hiding the subsystem:** In the strict GoF definition, the Facade can optionally hide the subsystem classes entirely, exposing only high-level methods. Our implementation exposes the individual services through getters (e.g., `getExpenseClientService()`) rather than wrapping their methods. This is a deliberate trade-off: wrapping every method of every service would create a monolithic class with dozens of delegating methods, contradicting the Single Responsibility Principle, while still providing no additional abstraction.

7.3.3 Application Launch

All JavaFX applications are required to have their entry point defined by a class which extends `javafx.Application`. The main method of the program has to call the static method `Application.launch()`: JavaFX then automatically calls the

`start` method, which has to be overridden in the entry point class. Here is part of HouseMate's client launcher class:

```
1
2  public class HouseMateApplication extends Application {
3
4      private final HttpClient httpClient = HttpClient.newHttpClient
       ();
5      private final ObjectMapper objectMapper = new ObjectMapper().
       registerModule(new JavaTimeModule());
6      private final SessionManager clientContext = new
       SessionManager(new AuthState());
7
8      private AppServices services;
9      private Stage primaryStage;
10
11     @Override
12     public void start(Stage primaryStage) {
13
14         this.primaryStage = primaryStage;
15         this.services = new AppServices(httpClient, objectMapper,
       clientContext);
16
17         //retrieve session data and validate it
18        //call to either showLoginScreen or showMainScreen
       depending on the saved session
19     }
20
21     public void logout(){
22         //clear session
23         showLoginScreen();
24     }
25
26     public void showLoginScreen() {
27         //load FXML
28         primaryStage.show();
29     }
30
31     public void showMainScreen() {
32         //load FXML
33         primaryStage.show();
34     }
35 }
```

Listing 7.2: Structure of the client module's entry point

This method is also the only place in the applications which instantiates the

AppServices, HttpClient, ObjectMapper and SessionManager dependencies and injects them down the Controller hierarchy by applying the controller factory technique on the MainController

7.3.4 Performing Backend Calls

Through the entire client module development, the fundamental rule of JavaFX had to be followed: *"Every action that interacts with JavaFX UI components must be performed on the JavaFX Application Thread"*. The JavaFX Application Thread is the main thread of every application build this way and all code runs by default on it. Because of that, if a backend call were to be performed within such thread and the code had to wait for the response (which can take a perceivable amount of time, depending on server latency and connection), the entire UI would be frozen until the response comes back because no UI-updating code could be executed. The necessary workaround consists of encapsulating the client service methods into a separate thread, with the native Java `CompletableFuture` module. When running any Java program, the JVW keeps a pool of background threads available for use. The static method `CompletableFuture.runAsync(() -> )` executes the provided `Runnable` on one of these threads. Running the backend calls with this construct allows for the UI to be responsive while waiting for a response from the backend.

```
ChoreCreateRequestDTO requestDTO = new ChoreCreateRequestDTO(
    txtChoreDesc.getText(),
    spnFrequencyDays.getValue(),
    services.getSessionManager().getCurrentHousehold().id()
);

CompletableFuture.runAsync(() -> {
    try {
        services.getChoreClientService().createChore(requestDTO);
        Platform.runLater(() -> {
            clearFields();
            mainController.showToast("Chore created successfully!"
   , MessageType.SUCCESS);
            onReturnCallback.run();
        });
    } catch (RuntimeException e) {
        Platform.runLater(() -> mainController.showToast(e.
   getMessage(), MessageType.ERROR));
    }
```

```
18 });
```

Listing 7.3: Core procedure required to perform a backend call

This snippet also shows the creation of the request DTO inside the Controller classes, which read the information provided by the user on the UI components (whenever more complex information is required to perform an action, such as expense creation, the data is not directly read from the JavaFX components but passed through temporary private variables) and perform the actual HTTP request by opportunely calling the client services. This enforces the separation of responsibilities applied to the project. Since all the code inside the `CompletableFuture` block is executed on a separate thread, the UI-updating code has to be run back on the JavaFX Application Thread, with its `Platform.runLater(() -> )` which simply executes the provided Runnable after the separate thread it's invoked in terminates.

7.4 UI/UX Functionalities

7.4.1 FXML Page Building and Loading

The core building blocks of the user interface are the static pages defined in FXML files. These files consist of JavaFX components (referred to as *nodes* in the JavaFX API) nested into each other to compose organized layouts. The following listing shows the structure of the main window:

```
1  <BorderPane prefHeight="750.0" prefWidth="420.0" styleClass="root"
       xmlns="http://javafx.com/javafx/21" xmlns:fx="http://javafx.
       com/fxml/1" fx:controller="com.housemate.client.controllers.
       MainController">
2      <top>
3          <!--Logo and "Refresh" button-->
4      </top>
5      <center>
6          <StackPane fx:id="outerContainer">
7              <StackPane fx:id="mainContentContainer"/>
8              <StackPane fx:id="popupLayer" mouseTransparent="true"/
       >
9              <StackPane fx:id = "confirmPopupLayer"
       mouseTransparent="true"/>
10         </StackPane>
11     </center>
12     <bottom>
13         <HBox alignment="CENTER" prefHeight="60.0" styleClass="nav
       -bar" BorderPane.alignment="CENTER">
```

```
14              <Button fx:id="btnNavH" maxWidth="Infinity" prefHeight
    ="60.0" styleClass="nav-button-active" HBox.hgrow="ALWAYS">
15                  <graphic>
16                      <Region styleClass="base-icon, household-nav-
    button"/>
17                  </graphic>
18              </Button>
19              <!--Three more exactly equal buttons for the other
    tabs-->
20          </HBox>
21      </bottom>
22  </BorderPane>
```

Listing 7.4: Core structure of the client's main window

The `fx:controller` attribute on the root node links each FXML file to its Controller class, which handles user interactions, invokes the client services, and updates the UI with backend data. The central content area deserves particular attention: the `outerContainer` is a `StackPane` that layers three children on top of each other. The `mainContentContainer` holds the currently active tab content, while `popupLayer` and `confirmPopupLayer` sit above it as transparent overlay planes. When a popup needs to be displayed, the `MainController` adds its node to the appropriate layer and toggles its visibility, making it appear on top of the main content. The `mouseTransparent="true"` attribute ensures that click events pass through the overlay layers to the content underneath when no popup is active.

Each FXML file is loaded programmatically through a `FXMLLoader`. By default, the JavaFX runtime instantiates the controller via its no-argument constructor, which makes dependency injection impossible. To work around this limitation, the application uses the `setControllerFactory` method, which replaces the default instantiation logic with a custom lambda that constructs the controller with the required dependencies:

```
1 FXMLLoader loaderAssignments = new FXMLLoader(getClass().
    getResource("/com/housemate/client/tabs/tab_assignments.fxml"))
    ;
2 loaderAssignments.setControllerFactory(clazz -> new
    TabAssignmentsController(this.services, this));
3 tabAssignments = loaderAssignments.load();
4 tabAssignmentsController = loaderAssignments.getController();
```

Listing 7.5: FXML loading with dependency injection via controller factory

This mechanism allows the `AppServices` Façade and the parent controller reference to be injected into every child controller at load time.

7.4.2 Hierarchical Structure of the Controllers

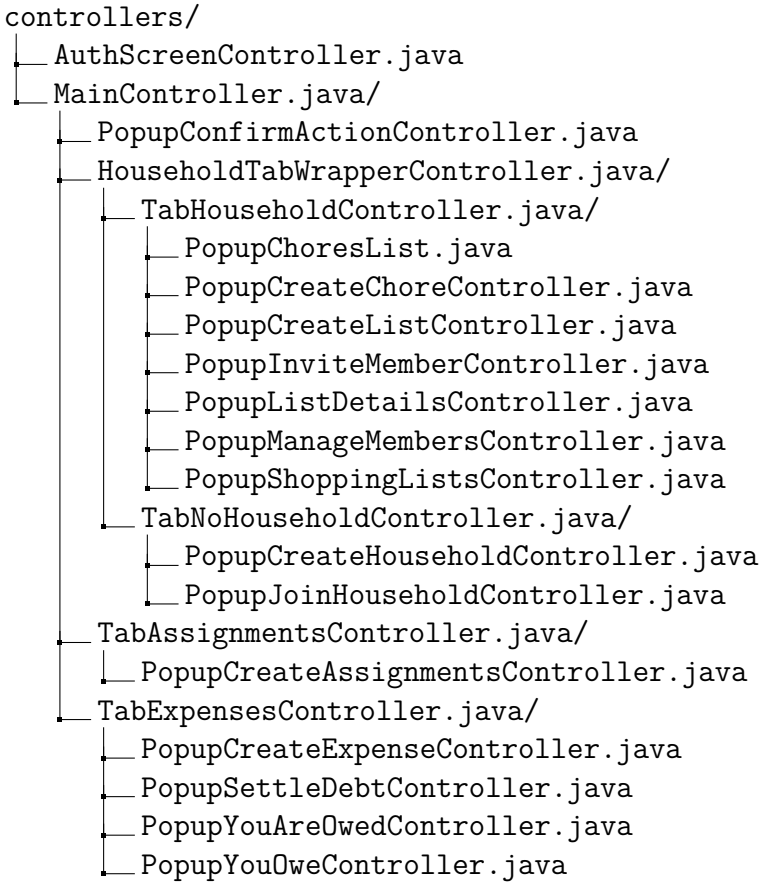

Figure 7.7: Complete structure of the Controller hierarchy

The Controller hierarchy mirrors the FXML file structure in a 1:1 relationship. When the `MainController` loads the four main tab FXML files, it passes itself (`this`) down to their controllers via the controller factory mechanism described above. Each tab controller, in turn, passes the `MainController` reference to the popup controllers it loads. This structure implements the Mediator design pattern: the `MainController` serves as a centralized coordinator for all cross-component interactions. Whenever a child controller needs to open a popup, display a toast notification, or trigger a tab switch, it calls a method on the `MainController` (e.g., `openPopup()`, `showToast()`, `switchTab()`). No child controller ever communicates directly with another child controller.

Toast notifications are transient messages displayed at the bottom of the screen to confirm successful operations or report errors. The `showToast(String message, MessageType type)` method on the `MainController` creates a styled label node, adds it to the overlay layer, and automatically removes it after a short delay.

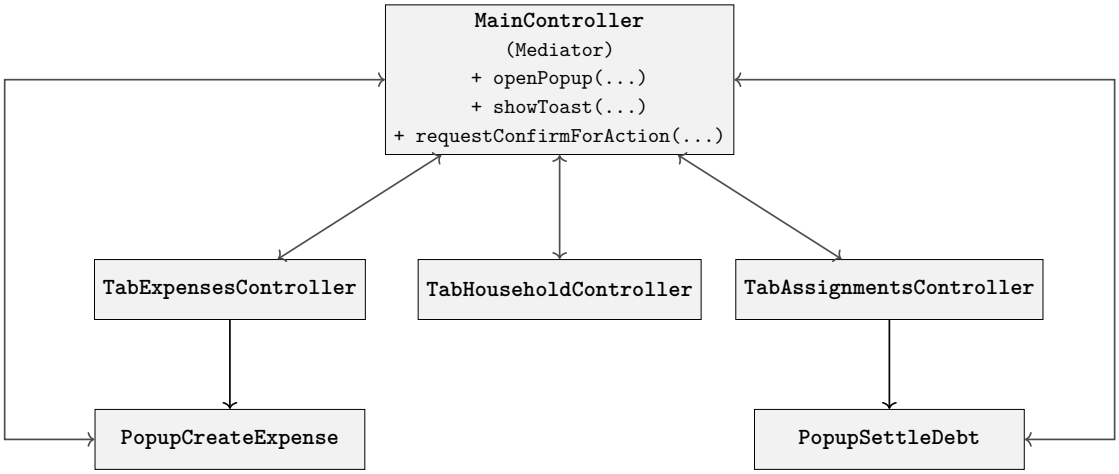

Figure 7.8: Mediator Pattern: Centralized UI Coordination

Comparison with the standard design pattern.

- **Concrete vs Abstract Mediator:** The GoF pattern defines the Mediator as an interface or abstract class, allowing multiple mediator implementations. Our application uses a single concrete `MainController` because the JavaFX client has exactly one main window with one coordination point. Introducing an abstract mediator would add unnecessary complexity without a realistic use case for substitution.
- **Bidirectional communication:** In the standard pattern, colleagues notify the mediator of events, and the mediator decides how to respond. Our implementation follows this closely: child controllers call the mediator's methods (upward direction), and the mediator manipulates the shared `StackPane` layers to display popups and toasts (downward direction). The child controllers never directly access the `popupLayer` or `confirmPopupLayer` nodes.
- **Decoupling benefit:** Without this mediator, each of the 15+ controllers would need references to every other controller it might interact with, creating a tightly coupled mesh of dependencies. The mediator reduces this to a star topology where each controller knows only the `MainController`.

7.4.3 Application Launch

Every JavaFX application requires an entry point class that extends `javafx.Application`. The program's `main` method calls `Application.launch()`, which triggers the framework to invoke the overridden `start` method. The following listing shows the structure of HouseMate's entry point:

```
public class HouseMateApplication extends Application {

    private final HttpClient httpClient = HttpClient.newHttpClient
    ();
    private final ObjectMapper objectMapper = new ObjectMapper().
    registerModule(new JavaTimeModule());
    private final SessionManager clientContext = new
    SessionManager(new AuthState());

    private AppServices services;
    private Stage primaryStage;

    @Override
    public void start(Stage primaryStage) {

        this.primaryStage = primaryStage;
        this.services = new AppServices(httpClient, objectMapper,
    clientContext);

        //retrieve session data and validate it
        //call to either showLoginScreen or showMainScreen
    depending on the saved session
    }

    public void logout(){
        //clear session
        showLoginScreen();
    }

    public void showLoginScreen() {
        //load FXML
        primaryStage.show();
    }

    public void showMainScreen() {
        //load FXML
        primaryStage.show();
    }
```

```
34 }
```

Listing 7.6: Structure of the client module's entry point

This class is the only place in the application that instantiates the `HttpClient`, `ObjectMapper`, `SessionManager`, and `AppServices` dependencies. These are then injected down the entire Controller hierarchy through the controller factory technique described in Section 7.4.1.

7.4.4 Dynamic UI Components Loading

While the static structure of each page is defined declaratively in FXML files, many parts of the user interface cannot be known at design time because they depend on data retrieved from the backend at runtime. Lists of expenses, household members, chores, debts and settlements are all examples of collections whose size and content change with every request. Rather than defining a fixed number of placeholder nodes in FXML, the application constructs these visual elements *programmatically* inside the Controller classes, following a single recurring archetype that is reused across every data-driven view.

Dedicated Component Classes

Every entity that requires a complex visual representation in a list, made out of many different JavaFX elements, owns a dedicated Java class inside the `components` package. Each class extends a JavaFX layout node (typically `HBox` or `VBox`) and receives the corresponding response DTO in its constructor, together with any contextual parameters needed for rendering (e.g., the current user's ID to determine the perspective, or callback functions for interactive buttons). The constructor reads the DTO fields and assembles the necessary JavaFX children (labels, icons, buttons, spacers) entirely in code, applying CSS style classes for consistent visual appearance. The full set of component classes in the application is:

- `ExpenseItemElement` – displays an expense's description, payer, date, total amount and the current user's share.
- `SettlementItemElement` – displays a settlement's description, counterpart, date and amount.
- `DebtItemElement` – displays a debt's counterpart, outstanding balance and, if the user is the debtor, a "Pay" button.

- `ChoreItemElement` – displays a chore's description, frequency and a "Delete" button.
- `MemberItemElement` – displays a household member's profile, payment link, and a "Remove" button.
- `MemberSplitBox` – an interactive component used during expense creation to configure each member's share.

The Fetch–Iterate–Build Archetype

Every Controller that needs to display a list of backend-provided entities follows the same three-step procedure:

1. **Clear the container.** The FXML file declares an empty `VBox` (the `dataContainer`) that serves as the target area for the dynamic content. Before every data refresh, the controller removes all existing children from this container to guarantee a clean state.
2. **Fetch the data asynchronously.** The controller calls the appropriate client service method inside a `CompletableFuture` (as described in Section 7.4.5) and obtains a list of response DTOs.
3. **Iterate and build.** Back on the JavaFX Application Thread (via `Platform.runLater`), the controller iterates over the DTO list and, for each element, instantiates the corresponding component class, passing the DTO and any required context. The newly created node is appended to the container's children list.

The following excerpt from the `TabExpensesController` illustrates this archetype for expense items:

```
dataContainer.getChildren().clear();   // Step 1: clear

// Step 2: fetch (already inside CompletableFuture)
List<?> transactions = services.getExpenseClientService()
    .getFilteredExpenses(filterRequestDTO);

Platform.runLater(() -> {              // Step 3: build
    for (var transaction : transactions) {
        ExpenseItemElement item = new ExpenseItemElement(
            (ExpenseResponseDTO) transaction,
            services.getSessionManager().getCurrentUser().id()
        );
        dataContainer.getChildren().add(item);
    }
```

```
});
```

Listing 7.7: Dynamic UI loading archetype (from TabExpensesController)

This pattern is applied identically in the `PopupManageMembersController` (with `MemberItemElement`), the `PopupChoresListController` (with `ChoreItemElement`), the `PopupYouOweController` and `PopupYouAreOwedController` (with `DebtItemElement`), and for settlement items within `TabExpensesController` (with `SettlementItemElement`). By encapsulating the rendering logic inside the component classes and keeping the iteration loop in the controllers, the application achieves a clean separation: components are responsible for *how* an entity looks, while controllers decide *when* and *which* entities to display.

7.4.5 Performing Backend Calls

Throughout the client module, the fundamental rule of JavaFX must be respected: *every action that interacts with UI components must be performed on the JavaFX Application Thread.* This is the main thread of every JavaFX application, and all code runs on it by default. If a backend call were executed on this thread, the UI would freeze until the response arrives, because no rendering or event-handling code could execute during the wait. The solution consists of offloading network calls to background threads using the native Java `CompletableFuture` API. The JVM maintains a pool of background threads available for asynchronous execution; the static method `CompletableFuture.runAsync()` dispatches the provided `Runnable` to one of these threads, keeping the UI responsive while the request is in flight.

```
ChoreCreateRequestDTO requestDTO = new ChoreCreateRequestDTO(
    txtChoreDesc.getText(),
    spnFrequencyDays.getValue(),
    services.getSessionManager().getCurrentHousehold().id()
);

CompletableFuture.runAsync(() -> {
    try {
        services.getChoreClientService().createChore(requestDTO);
        Platform.runLater(() -> {
            clearFields();
            mainController.showToast("Chore created successfully!"
   , MessageType.SUCCESS);
            onReturnCallback.run();
        });
    } catch (RuntimeException e) {
```

```
16          Platform.runLater(() -> mainController.showToast(e.
    getMessage(), MessageType.ERROR));
17      }
18  });
```

Listing 7.8: Core procedure for performing an asynchronous backend call

The listing also illustrates how request DTOs are assembled inside the Controller classes by reading values from the UI components. For operations requiring more complex input (such as expense creation with multiple split configurations), the data is first collected into temporary private variables rather than read directly from the JavaFX nodes at call time. Since all code inside the `CompletableFuture` block executes on a background thread, any subsequent UI updates—such as clearing input fields, displaying toast notifications, or triggering navigation callbacks—must be dispatched back to the JavaFX Application Thread via `Platform.runLater()`.

8. Testing and Quality Assurance

8.1 Testing Philosophy and Approach

In an application with the described articulated structure, software correctness cannot be inferred from visual inspection alone: security flaws or financial inaccuracies are categories of defects that remain invisible during development but can produce serious consequences in production, thus they must be prevented. To this end, a test-driven verification strategy was adopted. Every business-critical component is accompanied by automated tests that exercise happy-path behavior, validate edge cases, and ensure that invalid or adversarial inputs get rejected.

The structure of this testing landscape follows the *Test Pyramid* model, as described below:

1. **Unit Tests (Base Tier):** A broad base of fast, isolated tests that verify individual classes and methods;
2. **Repository Tests (Middle Tier):** A repository layer that exercises persistence against a real, in-memory database;
3. **Web-Layer Tests (Top Tier):** A specialized suite that validates HTTP semantics, authentication enforcement, and JSON contract compliance.

This layered approach ensures that smaller defects are caught as early and as cheaply as possible in the unit tests, while the integration tests guarantee that the application architecture remains stable at runtime.

8.2 Testing Technology Stack

Technology	Description and Role in the Test Suite
JUnit 5	The unit testing framework that allows to write and execute the tests, while also providing useful annotations such as `@Test`, `@Nested`, `@BeforeEach`, and `@DisplayName`, all widely used throughout the testing codebase.
Mockito	A mocking library used to create lightweight stubs of any component and infrastructure dependency in order to isolate the unit under test from external state.
AssertJ	An assertion library; preferred over JUnit's built-in assertions for its improved readability and strong support for `BigDecimal` comparison semantics, mainly needed to test the expense management module (see section 6.6).
Spring Boot Test	Provides the `@WebMvcTest` and `@DataJpaTest` test slices for controller and repository test classes. The benefit is that these annotations instruct the system to bootstrap only the Spring context layers required by the involved tests, minimizing startup and overall testing times.
Spring Security Test	Supplies the `@WithMockUser`, `@WithAnonymousUser`, and `csrf()` utilities used to simulate authenticated and unauthenticated HTTP requests in controller tests.
MockMvc	Spring's HTTP test client, which dispatches requests through the full DispatcherServlet pipeline (filters, serialization, exception handling) without starting a real web server.
H2 Database	Lightweight in-memory relational database that mirrors the PostgreSQL schema via Hibernate's `create-drop` DDL mode, providing a fresh state for every test execution.
TestEntityManager	*Spring Data JPA* utility that wraps the JPA `EntityManager`, offering `persistAndFlush` and `clear` methods for precise control over the persistence context in integration tests.

Table 8.1: Testing technology stack

8.3 Unit Testing

To illustrate the concrete application of our testing methodology, this section walks through specific unit test examples across the entire application stack. We will begin with basic function testing and gradually move up through the various tiers of the web architecture. Additionally, a dedicated subsection covers security testing, highlighting how unit tests can prevent vulnerabilities and enforce access control policies at the code level.

8.3.1 Pure Function Tests (Expense Split Strategies)

In the codebase, a perfect example of pure functions is identified by the expense splitting strategies. In fact, they receive two parameters (the total amount and the list of share requests) and return a deterministic map of user / share amount without holding any state. This makes them ideal candidates for exhaustive unit testing without any mocking. As an example, we present below one of the unit tests for the `EQUAL_SPLIT` strategy:

```
@Test
void calculateShares_withThreeUsersAndRemainder_distributesSinglePenny() {
    Map<UUID, BigDecimal> result = strategy.calculateShares(
            new BigDecimal("10.00"),
            List.of(
                    new ExpenseShareRequestDTO(u1, null),
                    new ExpenseShareRequestDTO(u2, null),
                    new ExpenseShareRequestDTO(u3, null)
            )
    );

    assertThat(result).hasSize(3);
    assertThat(sum(result)).isEqualByComparingTo("10.00");
    assertThat(result.values()).containsExactlyInAnyOrder(
            new BigDecimal("3.34"),
            new BigDecimal("3.33"),
            new BigDecimal("3.33")
    );
}
```

Listing 8.1: Example of a unit test for the Equal Split Strategy

While these constructs might appear straightforward to handle, actually they are the ones that need more in-depth analysis as they present a large number of edge cases and are the foundation on which the upper layers rely on.
In fact, sticking to the example, a critical property verified by every strategy test is the *sum invariant*: the sum of all computed shares must equal the original total exactly; this assertion catches rounding errors that would otherwise accumulate silently across hundreds of transactions. The strategies are also tested for rejection

of edge cases such as duplicate user IDs, null amounts, empty share lists, zero amounts, and negative amounts.

8.3.2 Service Layer Tests (Adding a Debt)

The backend service layer contains the most critical business logic, including the debt netting algorithm and the expense creation orchestration. These business logics are tested with Mockito-based unit tests that stub the repository layer, allowing the tests to run independently from the actual existence of a database. In the following snippet we provide the code of a unit test for the debt addition logic:

```
@Test
void addDebt_whenInverseDebtMatchesExactly_setsInverseDebtToZero() {
    BigDecimal amount = new BigDecimal("50.00");
    when(userRepository.findById(debtorId))
            .thenReturn(Optional.of(debtor));
    when(userRepository.findById(creditorId))
            .thenReturn(Optional.of(creditor));
    when(householdRepository.findById(householdId))
            .thenReturn(Optional.of(household));

    Debt inverseDebt = mock(Debt.class);
    when(inverseDebt.getAmount()).thenReturn(amount);
    when(debtRepository
            .findByDebtorAndCreditorAndHousehold(creditor, debtor, household))
            .thenReturn(Optional.of(inverseDebt));

    debtService.addDebt(debtorId, creditorId, householdId, amount);

    verify(inverseDebt).setAmount(BigDecimal.ZERO);
    verify(debtRepository).save(inverseDebt);
    verifyNoMoreInteractions(debtRepository);
}
```

Listing 8.2: Example of a unit test for the Debt Service

The `DebtServiceTest` class uses Mockito's `ArgumentCaptor` extensively to inspect the exact entities passed to `debtRepository.save()`, verifying not only that the correct method was called but that the saved entity carries the expected amount, debtor, creditor, and household. The test suite covers all five branches of the netting algorithm:

1. Debtor equals creditor (short-circuit, no repository calls).
2. No inverse debt exists: a new forward debt is created.
3. Inverse debt is greater than the new amount: inverse debt is reduced.
4. Inverse debt equals the new amount exactly: inverse debt is set to zero.

5. Inverse debt is smaller than the new amount: inverse debt is zeroed, and a forward debt is created for the remainder.

Each service test class also tests guard clauses: null user IDs, null filter DTOs, missing entities (user not found, household not found), and unauthorized state transitions (e.g. attempting to view debts without an active household membership). These tests use AssertJ's `assertThatThrownBy` to verify both the exception type and the error message.

8.3.3 Controller Layer Tests (Creating an Expense)

Controller tests use Spring's `@WebMvcTest` test slice, which bootstraps only the web layer and the security filter chain, while mocking all service dependencies. This approach verifies that endpoints:

- Return the correct HTTP status codes for success (e.g. 200 `OK`, 201 `CREATED`) and failure (e.g. 400 `BAD REQUEST`, 401 `UNAUTHORIZED`, 403 `FORBIDDEN`);
- Serialize response DTOs to the expected JSON structure;
- Reject invalid request payloads before they reach the service layer;
- Enforce authentication (via `@WithAnonymousUser`) and authorization correctly.

In the following example we provide a unit test for expense creation that handles both success and failure cases, ensuring the controller returns the correct status code in each situation and follows the correct behaviour both in terms of returned results and accessed services.

```
@WebMvcTest(ExpenseController.class)
@ContextConfiguration(classes = ServerApp.class)
@WithMockUser(username = "11111111-1111-1111-1111-111111111111")
class ExpenseControllerTest {

    @Test
    @DisplayName("POST /api/expenses - returns 201 with created expense")
    void testCreateExpense_Success() throws Exception {
        when(expenseService.createExpense(eq(TEST_USER_UUID), any()))
                .thenReturn(expenseResponse);

        mockMvc.perform(post(BASE_URL)
                    .with(csrf())
                    .contentType("application/json")
                    .content(objectMapper.writeValueAsString(validCreateRequest)))
                .andExpect(status().isCreated())
                .andExpect(jsonPath("$.id").value(EXPENSE_ID))
                .andExpect(jsonPath("$.description").value("Groceries"))
                .andExpect(jsonPath("$.amount")
```

```
                        .value(comparesEqualTo(
                            new BigDecimal("120.00")), BigDecimal.class
                        ))
                .andExpect(jsonPath("$.splitType")
                        .value(ExpenseSplitType.EQUAL_SPLIT.name()));

        verify(expenseService).createExpense(eq(TEST_USER_UUID), any());
    }

    @Test
    @WithAnonymousUser
    @DisplayName("POST /api/expenses - returns 401 for unauthenticated user")
    void testCreateExpense_Unauthenticated() throws Exception {
        mockMvc.perform(post(BASE_URL).with(csrf())
                        .contentType("application/json")
                        .content(objectMapper.writeValueAsString(
                            validCreateRequest
                        )))
                .andExpect(status().isUnauthorized());

        verify(expenseService, never()).createExpense(any(), any());
    }
}
```

Listing 8.3: Example of a unit test for the Expense Controller

The use of `@WithMockUser` and `@WithAnonymousUser` annotations allows the test suite to verify the entire Spring Security filter chain end-to-end, including the `JwtAuthenticationFilter`, without constructing actual JWT tokens. The `csrf()` processor is applied to all state-changing requests to comply with Spring Security's CSRF protection.

8.3.4 Client Service Tests

Each `ClientService` test class mocks the `HttpRestClient` helper to simulate DTOs serialization, JSON deserialization and server responses, then verifies that:

1. HTTP requests are constructed with the correct method, URI path, query parameters, content type, and `Authorization` header.
2. Successful responses (status 2xx) are correctly deserialized into the expected DTO types.
3. Error responses produce a `RuntimeException` containing the expected status code and server message.

```
@Test
@DisplayName("createExpense - should successfully create an expense")
void testCreateExpense_Success() throws Exception {
    String jsonResponse = objectMapper.writeValueAsString(testExpenseResponseDTO);
    HttpResponse<String> mockResponse = createMockResponse(201, jsonResponse);
```

```
6
7       when(mockHttpRestClient.serializeDTO(any())).thenReturn("{\"description\":\"
        Groceries\"}");
8       when(mockHttpRestClient.sendRequest(any(HttpRequest.class)))
9               .thenReturn(mockResponse);
10      when(mockHttpRestClient.deserializeDTO(
11                      jsonResponse, ExpenseResponseDTO.class
12              )
13          )
14              .thenReturn(testExpenseResponseDTO);
15      when(mockHttpRestClient.buildAuthHeader())
16              .thenReturn("Bearer test-token");
17
18      ExpenseResponseDTO result = expenseClientService
19              .createExpense(testExpenseCreateRequestDTO);
20
21      assertNotNull(result);
22      assertEquals(TEST_EXPENSE_ID, result.id());
23      assertEquals(TEST_DESCRIPTION, result.description());
24
25      verify(mockHttpRestClient).serializeDTO(any());
26      ArgumentCaptor<HttpRequest> requestCaptor = ArgumentCaptor.forClass(
27          HttpRequest.class
28      );
29      verify(mockHttpRestClient).sendRequest(requestCaptor.capture());
30      verify(mockHttpRestClient).deserializeDTO(
31          jsonResponse, ExpenseResponseDTO.class
32      );
33      verify(mockHttpRestClient).buildAuthHeader();
34
35      HttpRequest capturedRequest = requestCaptor.getValue();
36      assertEquals("POST", capturedRequest.method());
37      assertTrue(capturedRequest.uri().getPath().endsWith("/api/expenses"));
38      assertEquals("application/json", capturedRequest.headers()
39          .firstValue("Content-Type").orElse(null)
40      );
41      assertEquals("Bearer test-token", capturedRequest.headers()
42          .firstValue("Authorization").orElse(null)
43      );
44  }
```

Listing 8.4: Example of a unit test for the Expense Client Service

The use of `ArgumentCaptor` on the `HttpRequest` object is a key technique: it allows the test to inspect the exact URI, HTTP method, headers, and content type of the request that the service would send to the real server, guaranteeing that the client and server agree on the API contract.

8.3.5 Security Layer Tests

Given the sensitivity of authentication mechanisms, the security filters require dedicated unit tests to guarantee that invalid, expired, or missing credentials are appropriately managed without degrading into server errors. The `JwtAuthenticationFilterTest`

achieves this by isolating the filtering logic from the web application utilizing `MockHttpServletRequest`, `MockHttpServletResponse`, and `MockFilterChain`.

By mocking the underlying `JwtService` and `UserService`, the test suite systematically simulates various edge cases. These include requests with no `Authorization` header, tokens with an invalid UUID format, and tokens triggering exceptions. A critical requirement verified by these tests is that token parsing errors (such as `ExpiredJwtException` or `MalformedJwtException`) do not propagate as internal server errors (HTTP 500). Instead, the filter safely aborts the authentication process while allowing the filter chain to continue, gracefully delegating the rejection logic to the downstream security components:

```
@Test
void expiredToken_jwtExceptionCaught_filterPassesThrough_noAuthentication() throws
     Exception {
    String token = "expired.jwt.token";
    MockHttpServletRequest request = new MockHttpServletRequest();
    request.addHeader("Authorization", "Bearer " + token);
    MockHttpServletResponse response = new MockHttpServletResponse();
    MockFilterChain chain = new MockFilterChain();

    when(jwtService.extractSubject(token))
            .thenThrow(new ExpiredJwtException(null, null, "Token expired"));

    jwtAuthenticationFilter.doFilterInternal(request, response, chain);

    assertThat(SecurityContextHolder.getContext().getAuthentication()).isNull();
    assertThat(response.getStatus()).isEqualTo(200); // Filter chain continues
    downstream without crashing
}
```

Listing 8.5: Example of a unit test verifying expired token handling in the JWT Filter

This approach guarantees that the security context (`SecurityContextHolder`) is only populated upon successful validation of a legitimate token, effectively securing the application state right at the entry point of the filter chain.

8.3.6 Global Exception Handler Tests

To ensure that business logic constraints and unexpected errors are consistently translated into robust HTTP responses, the application uses a centralized `@RestControllerAdvice`. The `GlobalExceptionHandlerTest` suite verifies this component by directly invoking its handler methods with various mocked or instantiated exceptions.

These unit tests confirm that specific exception types map to the correct HTTP status codes and payload formats as described in section **??**. Notably, a strong emphasis is placed on ensuring complex validation errors, which throw `BindException`,

are handled reliably. The tests guarantee that the handler correctly extracts multiple field errors, aggregates them into a structured key-value map, and gracefully falls back to default messages such as `"Invalid value"` when specific error text is missing:

```
@Test
void handleValidationExceptions_methodArgumentNotValid_multipleErrors() {
    MethodArgumentNotValidException exception = mock(
        MethodArgumentNotValidException.class
    );
    BindingResult bindingResult = mock(BindingResult.class);
    List<ObjectError> errors = new ArrayList<>();
    errors.add(new FieldError("objectName", "email", "Invalid email format"));
    errors.add(new FieldError(
            "objectName", "password", "Password must be at least 8 characters"
        )
    );
    errors.add(new FieldError("objectName", "name", "Name cannot be blank"));

    when(exception.getBindingResult()).thenReturn(bindingResult);
    when(bindingResult.getAllErrors()).thenReturn(errors);

    ResponseEntity<Map<String, String>> response =
            globalExceptionHandler.handleValidationExceptions(exception);

    assertThat(response.getStatusCode()).isEqualTo(HttpStatus.BAD_REQUEST);
    assertThat(response.getBody())
            .containsEntry("error", "Validation Failed")
            .containsEntry("email", "Invalid email format")
            .containsEntry("password", "Password must be at least 8 characters")
            .containsEntry("name", "Name cannot be blank");
}
```

Listing 8.6: Example of a unit test for validation error handling

This comprehensive verification guards against internal stack traces leaking to the client layer and enforces a predictable, client-friendly error reporting contract across the application's REST APIs.

8.4 Integration Testing

While unit tests verify individual components in isolation, integration tests verify that multiple layers collaborate correctly when wired together by the Spring context. Three integration test classes use the `@DataJpaTest` slice combined with `@Import` to load the real service classes alongside an H2 in-memory database, while mocking only the components that are not relevant to the test scenario.

8.4.1 Expense Creation Integration Test

The `ExpenseServiceIntegrationTest` verifies the full transactional flow of expense creation: persisting the expense entity, creating the expense shares, and generating the corresponding debt records. The test persists real `User`, `Household`, and `HouseholdMembership` entities via the `TestEntityManager`, then invokes `expenseService.createExpense` and asserts that the database contains the expected expense shares and debt rows.

```
@Test
@DisplayName("createExpense persists expense graph and creates debt rows")
void createExpense_persistsExpenseAndCreatesDebtsForNonPayerShares() {
    Household household = persistHousehold("Main Household");
    User payer = persistUser("Alice", "Payer", "alice@test.com");
    User debtor = persistUser("Bob", "Debtor", "bob@test.com");
    persistMembership(household, payer, true);
    persistMembership(household, debtor, false);

    // ... strategy mock configuration ...

    ExpenseResponseDTO result = expenseService.createExpense(
            payer.getId(), request);

    assertThat(result.id()).isNotNull();
    assertThat(result.shares()).hasSize(2);
    assertThat(debtRepository.findAll())
        .hasSize(1)
        .first()
        .satisfies(savedDebt -> {
            assertThat(savedDebt.getDebtor().getId()).isEqualTo(debtor.getId());
            assertThat(savedDebt.getCreditor().getId()).isEqualTo(payer.getId());
            assertThat(savedDebt.getAmount()).isEqualByComparingTo("20.00");
        });
}
```

Listing 8.7: Verifying the full expense creation transaction (ExpenseServiceIntegrationTest)

8.4.2 Debt Netting Integration Test

The `DebtServiceIntegrationTest` exercises the netting algorithm against a real H2 database, verifying three critical scenarios:

1. When a new debt is smaller than an existing inverse debt, the inverse debt is reduced and no new row is created.
2. When a new debt exceeds an existing inverse debt, the inverse debt is set to zero and a forward remainder is created as a separate row.
3. When a new debt exactly cancels an existing inverse debt, the inverse debt's amount is set to zero and the row is preserved (not deleted), confirming the zero-persistence design decision described in the backend chapter.

8.4.3 Settlement Integration Test

The `SettlementServiceIntegrationTest` verifies that settling a debt correctly persists the `Settlement` entity and reduces the `Debt` balance within a single transaction. Two scenarios are tested: a partial settlement that reduces the debt from 100.00 to 60.00, and a full settlement that sets the debt to exactly zero without deleting the debt row.

8.5 Complete Test Inventory

The following table provides a complete inventory of all test classes in the codebase, organized by module and architectural layer. The two modules, `backend` and `client`, are presented separately.

Layer	Test Class	Test Type
Backend Module		
Config / Security	`GlobalExceptionHandlerTest`	Unit
	`JwtAuthenticationFilterTest`	Unit
Controller	`AuthControllerTest`	Web Slice
	`ChoreControllerTest`	Web Slice
	`DebtControllerTest`	Web Slice
	`ExpenseControllerTest`	Web Slice
	`HouseholdControllerTest`	Web Slice

Continued on next page

Layer	Test Class	Test Type
	SettlementControllerTest	Web Slice
	ShoppingListControllerTest	Web Slice
	UserControllerTest	Web Slice
Repository	ChoreAssignmentRepositoryTest	Data Slice
	DebtRepositoryTest	Data Slice
	DebtSpecificationTest	Data Slice
	ExpenseRepositoryTest	Data Slice
	ExpenseSpecificationTest	Data Slice
	HouseholdRepositoryTest	Data Slice
	QuerySpecificationTest	Data Slice
	SettlementRepositoryTest	Data Slice
	SettlementSpecificationTest	Data Slice
	UserRepositoryTest	Data Slice
Service	AuthServiceTest	Unit
	ChoreServiceTest	Unit
	DebtServiceTest	Unit
	ExpenseServiceTest	Unit
	HouseholdServiceTest	Unit
	JwtServiceTest	Unit
	SettlementServiceTest	Unit
	ShoppingListServiceTest	Unit
	UserServiceTest	Unit
Strategy	AdjustmentStrategyTest	Unit
	EqualSplitStrategyTest	Unit
	ExactAmountStrategyTest	Unit
	ExpenseSplitStrategyFactoryTest	Unit
	SharesSplitStrategyTest	Unit
Integration	DebtServiceIntegrationTest	Integration
	ExpenseServiceIntegrationTest	Integration
	SettlementServiceIntegrationTest	Integration
Client Module		
Client Service	AuthClientServiceTest	Unit
	ChoreClientServiceTest	Unit
	DebtClientServiceTest	Unit

Continued on next page

Layer	Test Class	Test Type
	ExpenseClientServiceTest	Unit
	HouseholdClientServiceTest	Unit
	HttpRestClientTest	Unit
	SettlementClientServiceTest	Unit
	ShoppingListClientServiceTest	Unit
	UserClientServiceTest	Unit

In total, the test suite comprises **48 test classes** spanning 6 architectural layers across 2 modules. The backend module accounts for 39 test classes and the client module contributes 9 test classes, each mirroring a corresponding client service.

8.6 Test Environment Configuration

The test environment is isolated from production through a dedicated Spring profile. The `application-test.properties` file configures:

```
jwt.secret=<TEST_PROFILE_JWT_SECRET>
jwt.expiration-ms=86400000

# H2 in-memory database
spring.jpa.hibernate.ddl-auto=create-drop
```

Listing 8.8: Test environment configuration

The `create-drop` DDL strategy guarantees that Hibernate generates the complete schema from the entity annotations at startup and destroys it at shutdown, providing a clean database for every test execution. The H2 dependency is scoped to `runtime` in the backend's `pom.xml`, ensuring it is available during testing but excluded from the production deployment artifact, where PostgreSQL is used instead.

The `@DataJpaTest` annotation activates this configuration automatically for repository and integration tests, providing:

- Automatic rollback after each test method, preventing state leakage between tests.
- A `TestEntityManager` bean for fine-grained control over entity persistence and cache clearing.

- Exclusion of non-JPA Spring components (web controllers, security filters, etc.), keeping the test context minimal and fast.

9. Project Management and Conclusion

9.1 Project Summary

The HouseMate project proved to be a substantive undertaking that solidified concepts embedded deeply within complex software engineering. We transitioned from initial requirement elicitations all the way through physical deployment, maintaining rigorous methodologies surrounding database schema creation, client-server REST paradigms, and security enforcement.

Automating chore tracking, household finances, and shared coordination reduces cognitive overhead for residents and tackles a real-world friction point efficiently. By building separate and highly decoupled Java components (a Spring Boot backend, a Java client, and shared DTOs), we ensured robust extensibility.

9.2 Challenges and Learnings

Several challenges were encountered and appropriately solved during the execution phase:

1. **Object-Relational Mapping (ORM) Complexities:** Translating arbitrary financial debts connecting User instances across an arbitrary Expense record proved to be challenging due to circular relational logic constraints. Breaking this down via specific bridging tables (`ExpenseShare`) established necessary clarity.
2. **Authentication Logistics:** Passing context implicitly throughout the web-filter chains leveraging JWTs required comprehensive thread-safe considerations in Spring contexts. Understanding the `SecurityContextHolder` allowed for proper authorization strategies.
3. **Dependency Scope Isolations:** Utilizing a multi-modules Maven project required understanding exactly which libraries must trace dependencies backwards into the `Shared` module without creating circular dependencies.

9.3 Future Work

Although the application achieves all MVP (Minimum Viable Product) objectives, logical extensions for scaling the ecosystem include:

- **Mobile Application Support:** Since the Backend is completely separated from the Frontend, integrating a Kotlin (Android) or Swift (iOS) client requires zero backend logic changes, opening the ecosystem up completely to portable devices.
- **Push Notifications:** Alerting household members asynchronously when new debts originate or when chores are overdue via WebSockets or Firebase Integration.
- **Payment Gateway Integrations:** Facilitating absolute real-world financial settlement integrations via Stripe APIs or PayPal architectures instead of relying solely on tracking internal ledger variables.